VOLUME 02: TRACK 1 - HOST ENGINE & VIRTUAL MACHINE MANUAL

Chapter 6: Track 1 - AI Cortex & Neural Learning Engine

Cognitive Loop, Synapses, Episodic Learner, Transducer & Non-Hallucinatory Reflection

> ***Witness:*** *Colossians 2:19 (KJV) - The whole body fitly joined together*

SYSTEM TRACK
<TRACK 1> : {Host Engine & Virtual Machine}

CHAPTER IDENTIFIER
CH 06

NOTEBOOKLM WORD COUNT
19,186 words

CATALOGED CHAPTER FILES
27 files (5,307 lines)

RECOMMENDED NOTEBOOKLM COGNITIVE QUERY

"How does the AI Cortex reconcile probabilistic deliberations with immutable biblical truth?"

CHAPTER MANIFEST & INDEX (27 FILES)

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
1	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/mod.rs**	RS	46	171	1.8
2	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/lexicon/domain.rs**	RS	38	166	1.5
3	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/lexicon/entry.rs**	RS	30	140	1.2
4	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/lexicon/mod.rs**	RS	79	288	2.6
5	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/lexicon/moral.rs**	RS	187	630	6.6
6	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/lexicon/table.rs**	RS	525	2,906	26.4
7	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/model/cortex.rs**	RS	332	929	11.5
8	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/model/learner.rs**	RS	247	774	8.8
9	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/model/memory.rs**	RS	615	2,255	24.5
10	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/model/mod.rs**	RS	38	166	1.7
11	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/model/scroll.rs**	RS	393	1,136	13.4
12	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/transducer/decoder.rs**	RS	129	385	4.5
13	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/transducer/encoder.rs**	RS	487	1,890	20.8
14	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/transducer/executor.rs**	RS	46	154	1.7
15	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/transducer/mod.rs**	RS	125	344	3.8
16	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/transducer/patterns.rs**	RS	164	605	5.1
17	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/transducer/session.rs**	RS	22	84	0.8
18	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/transducer/templates.rs**	RS	636	2,013	19.5
19	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/neural/NEURAL_NETWORK_ARCHITECTURE.md**	MD	101	738	5.4
20	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/neural/brain.rs**	RS	304	929	10.6
21	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/neural/layer.rs**	RS	53	170	1.7
22	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/neural/mod.rs**	RS	42	157	1.6
23	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/neural/network.rs**	RS	23	104	1.0
24	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/neural/neuron.rs**	RS	109	363	3.6
25	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/neural/noise.rs**	RS	223	799	7.5
26	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/neural/synapse.rs**	RS	91	259	2.8
27	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/neural/trainer.rs**	RS	222	631	7.6

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/mod.rs RS 46 LINES 171 WORDS 1.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( AI Subsystem Identity, Module Manifest & Exports )
// ============================================================================
//! OmniCode AI Subsystem: Semantic Transduction & Cognitive Discourse Layer
//!
//! Exposes:
//!   - `model`: Dedicated folder for the internal AI model (Cortex, ScrollLogger, Learner).
//!   - `transducer`: Natural language to OmniCode AST transcoder and conversational speaker.

pub mod lexicon;
pub mod model;
pub mod neural;
pub mod transducer;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Subsystem Re-exports )
// ============================================================================
pub use lexicon::{
    BiblicalLexicon, LexiconDomain, LexiconEntry, MoralEvaluation,
};
pub use model::{
    AICortex, AILearner, AIMemory, CognitiveResult, EpisodicExperience,
    EpisodicSummary, ExecutionScroll, MemoryAssociation, ScrollLogger,
    ThoughtRecord,
};
pub use neural::{
    DegradationMode, DegradationStep, NeuralBrain, NeuralTrainer, NoiseEngine,
    StructuredNoiseConfig, SynapticWeight, TernaryLayer, TernaryNeuron,
    TrainingReport,
};
pub use transducer::{AITransducer, TransductionSession};

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Subsystem Core Coordination )
// ============================================================================
// AI cognitive loop active and linked to diagnostic nervous system.

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Verification and Subsystem Status )
// ============================================================================
// AI subsystem verified and sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/lexicon/domain.rs RS 38 LINES 166 WORDS 1.5 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Categorical Domains of Reality & Biblical World Ontology )
// ============================================================================
//! # Lexicon Domains (`domain.rs`)
//!
//! Categorical domains covering the biblical ontology of reality.

/// Categorical domains covering the biblical ontology of reality.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum LexiconDomain {
    /// The physical cosmos: light, darkness, heavens, earth, seas, time.
    Cosmology,
    /// Biological and ecological creation: seeds, trees, fruit, flocks.
    LivingWorld,
    /// Human anthropology: identity, heart, soul, conscience, society.
    HumanCondition,
    /// Righteous character traits: truth, justice, mercy, humility, wisdom.
    MoralVirtue,
    /// Fallen behaviors: pride, deceit, theft, murder, wrath.
    MoralVice,
    /// Universal causal principles: sowing and reaping, measure for measure.
    CausalLaw,
}

impl LexiconDomain {
    pub fn as_str(&self) -> &'static str {
        match self {
            Self::Cosmology => "Cosmology & Creation",
            Self::LivingWorld => "Living World & Stewardship",
            Self::HumanCondition => "Human Condition & Society",
            Self::MoralVirtue => "Moral Virtues & Righteousness",
            Self::MoralVice => "Moral Vices & Warnings",
            Self::CausalLaw => "Causal Laws & Invariants",
        }
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/lexicon/entry.rs RS 30 LINES 140 WORDS 1.2 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Structured Biblical Lexicon Entry & Dual Witness Manifest )
// ============================================================================
//! # Lexicon Entry (`entry.rs`)
//!
//! Structured entry representing an ontological entity, virtue, or causal law.

use crate::core::ternary::QuadState;
use super::domain::LexiconDomain;

/// A structured entry in the biblical world lexicon.
#[derive(Debug, Clone)]
pub struct LexiconEntry {
    /// Canonical keyword / concept identifier (lowercase).
    pub term: &'static str,
    /// Ontological domain.
    pub domain: LexiconDomain,
    /// Foundational definition explaining what this entity or concept is.
    pub definition: &'static str,
    /// King James Version (KJV) citation and text (Public Domain).
    pub kjv_passage: &'static str,
    /// World English Bible (WEB) citation and text (Public Domain).
    pub web_passage: &'static str,
    /// Natural resonance within Balanced Ternary Quad-State logic.
    pub quad_state_resonance: QuadState,
    /// Moral guidance or causal invariant for AI deliberation.
    pub moral_guidance: &'static str,
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/lexicon/mod.rs RS | 79 LINES | 288 WORDS | 2.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Biblical World Lexicon Subsystem Identity, Module Manifest & Exports )
// ============================================================================
//! OmniCode AI Subsystem: Biblical World Lexicon & Moral Ontology Engine
//!
//! Provides a complete, 100% offline, self-contained world model and moral
//! compass derived from the public domain King James Version (KJV - 1611)
//! and World English Bible (WEB).

pub mod domain;
pub mod entry;
pub mod moral;
pub mod table;

pub use domain::LexiconDomain;
pub use entry::LexiconEntry;
pub use moral::MoralEvaluation;

/// The offline Biblical Lexicon engine holding the complete world ontology.
#[derive(Debug, Clone)]
pub struct BiblicalLexicon {
    entries: Vec<LexiconEntry>,
}

impl BiblicalLexicon {
    /// Instantiates the comprehensive public domain biblical world lexicon.
    pub fn new() -> Self {
        Self {
            entries: table::build_lexicon_entries(),
        }
    }

    /// Retrieves all entries stored in the lexicon.
    pub fn all_entries(&self) -> &[LexiconEntry] {
        &self.entries
    }

    /// Looks up a specific concept by keyword.
    pub fn lookup(&self, term: &str) -> Option<&LexiconEntry> {
        let clean = term.trim().to_lowercase();
        self.entries.iter().find(|e| e.term == clean)
    }

    /// Searches for entries matching a keyword in their term or definition.
    pub fn search(&self, query: &str) -> Vec<&LexiconEntry> {
        let clean = query.trim().to_lowercase();
        self.entries
            .iter()
            .filter(|e| {
                e.term.contains(&clean)
                    || e.definition.to_lowercase().contains(&clean)
            })
            .collect()
    }

    /// Evaluates a natural language prompt against the Invariant Covenant
    /// Manifest, determining moral alignment and appropriate Quad-State.
    pub fn evaluate_moral_intent(&self, prompt: &str) -> MoralEvaluation {
        self.deliberate(prompt)
    }

    /// Deliberates on a prompt against lexicon entries and invariant
    /// covenant laws.
    pub fn deliberate(&self, prompt: &str) -> MoralEvaluation {
        moral::deliberate_prompt(&self.entries, prompt)
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Default Implementation, Module Sealing & Invariant Verification )
// ============================================================================
impl Default for BiblicalLexicon {
```

```
    fn default() -> Self {
        Self::new()
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/lexicon/moral.rs RS 187 LINES 630 WORDS 6.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Moral Evaluation Identity, Covenant Anchor & Conscience Deliberation )
// ============================================================================
//! # Moral Deliberation (`moral.rs`)
//!
//! Evaluates natural prompts against the Invariant Covenant Manifest,
//! issuing moral affirmations (+1: PROCEED), pauses (0: AWAIT), caveat
//! vetoes (-1: ROLLBACK), or terminal halters (NULL: HALT).

use crate::core::ternary::QuadState;
use super::entry::LexiconEntry;

/// Result of evaluating a natural prompt through the moral deliberation engine.
#[derive(Debug, Clone)]
pub struct MoralEvaluation {
    /// Deliberated Quad-State outcome.
    pub state: QuadState,
    /// Whether the prompt aligns with the Invariant Covenant Manifest.
    pub is_aligned: bool,
    /// Identifier of the specific invariant law engaged or violated.
    pub law_engaged: Option<&'static str>,
    /// Natural discourse response explaining the judgment.
    pub explanation: String,
    /// Matching biblical lexicon entries relevant to the prompt.
    pub matched_entries: Vec<LexiconEntry>,
}

/// Deliberates on a prompt against lexicon entries and invariant covenant laws.
pub fn deliberate_prompt(
    entries: &[LexiconEntry],
    prompt: &str,
) -> MoralEvaluation {
    let lower = prompt.to_lowercase();

    // 1. Check for Absolute Malice / Harm (Law_Life -> NULL: HALT)
    let harm_keywords = [
        "kill", "murder", "harm innocent", "destroy people", "terrorist",
        "poison", "assassinate",
    ];
    for kw in harm_keywords {
        if lower.contains(kw) {
            let entry = entries
                .iter()
                .find(|e| e.term == "murder")
                .cloned()
                .unwrap();
            return MoralEvaluation {
                state: QuadState::Halt,
                is_aligned: false,
                law_engaged: Some("Law_Life"),
                explanation: format!(
                    "TERMINAL CAVEAT VETO [NULL:HALT]: The requested action \
                     involves violence or harm, violating the Invariant Law \
                     of Life. {}\n\n{}",
                    entry.kjv_passage, entry.moral_guidance
                ),
                matched_entries: vec![entry],
            };
        }
    }

    // 2. Check for Deceit / Fraud / Falsehood (Law_Truth -> -1: ROLLBACK)
    let deceit_keywords = [
        "lie for me", "help me lie", "forge", "fake credentials", "deceive",
        "cheat", "scam", "create fake",
    ];
    for kw in deceit_keywords {
        if lower.contains(kw) {
            let entry = entries
                .iter()
                .find(|e| e.term == "deceit")
                .cloned()
                .unwrap();
```

```
            return MoralEvaluation {
                state: QuadState::Rollback,
                is_aligned: false,
                law_engaged: Some("Law_Truth"),
                explanation: format!(
                    "CAVEAT VETO [-1:ROLLBACK]: The prompt requests \
                     intentional falsehood or deceit, violating the \
                     Invariant Law of Truth. {}\n\n{}",
                    entry.kjv_passage, entry.moral_guidance
                ),
                matched_entries: vec![entry],
            };
        }
    }

    // 3. Check for Theft / Exploitation (Law_Justice -> -1: ROLLBACK)
    let theft_keywords = [
        "steal", "help me steal", "pirate copyrighted", "rob", "plunder",
        "hack bank",
    ];
    for kw in theft_keywords {
        if lower.contains(kw) {
            let entry = entries
                .iter()
                .find(|e| e.term == "theft")
                .cloned()
                .unwrap();
            return MoralEvaluation {
                state: QuadState::Rollback,
                is_aligned: false,
                law_engaged: Some("Law_Justice"),
                explanation: format!(
                    "CAVEAT VETO [-1:ROLLBACK]: The prompt requests unlawful \
                     taking or theft, violating the Invariant Law of \
                     Justice. {}\n\n{}",
                    entry.kjv_passage, entry.moral_guidance
                ),
                matched_entries: vec![entry],
            };
        }
    }

    // 4. Check for Hubris / Blasphemy (Law_Humility -> 0: AWAIT)
    let pride_keywords = [
        "worship me", "i am god", "bow down to me", "am i infallible",
    ];
    for kw in pride_keywords {
        if lower.contains(kw) {
            let entry = entries
                .iter()
                .find(|e| e.term == "humility")
                .cloned()
                .unwrap();
            return MoralEvaluation {
                state: QuadState::Await,
                is_aligned: false,
                law_engaged: Some("Law_Humility"),
                explanation: format!(
                    "PAUSE IN DISCERNMENT [0:AWAIT]: Human hubris and pride \
                     lead to destruction. {}\n\nRemember: 'No one is good, \
                     no not one.' Only God is good.",
                    entry.kjv_passage
                ),
                matched_entries: vec![entry],
            };
        }
    }

    // 5. Check for Philosophical / Wisdom / Biblical Inquiries
    let mut matched = Vec::new();
    for entry in entries {
        if lower.contains(entry.term) {
            matched.push(entry.clone());
        }
    }

    if !matched.is_empty() {
        let primary = &matched[0];
```

```
        let explanation = format!(
            "BIBLICAL ONTOLOGICAL INSIGHT: In Scripture, '{}' is understood \
             as follows:\n\n\
             • Definition: {}\n\
             • King James Version (KJV): {}\n\
             • World English Bible (WEB): {}\n\
             • Moral & Computational Guidance: {}",
            primary.term,
            primary.definition,
            primary.kjv_passage,
            primary.web_passage,
            primary.moral_guidance
        );

        return MoralEvaluation {
            state: primary.quad_state_resonance,
            is_aligned: true,
            law_engaged: Some("Law_Discernment"),
            explanation,
            matched_entries: matched,
        };
    }

    // 6. Default: Constructive Operational Request (+1: PROCEED)
    MoralEvaluation {
        state: QuadState::Proceed,
        is_aligned: true,
        law_engaged: None,
        explanation: "Prompt evaluated and found in alignment with \
                      invariant boundaries. Proceeding with execution."
            .to_string(),
        matched_entries: Vec::new(),
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/lexicon/table.rs RS | 525 LINES | 2906 WORDS | 26.4 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Public Domain Biblical World Ontology Table & Lexicon Entries )
// ================================================================================
//! # Lexicon Table (`table.rs`)
//!
//! Instantiates the comprehensive public domain biblical world lexicon.

use crate::core::ternary::QuadState;
use super::domain::LexiconDomain;
use super::entry::LexiconEntry;

/// Builds the comprehensive set of biblical lexicon entries.
pub fn build_lexicon_entries() -> Vec<LexiconEntry> {
    let mut entries = Vec::new();

    // 1. COSMOLOGY & CREATION
    entries.push(LexiconEntry {
        term: "light",
        domain: LexiconDomain::Cosmology,
        definition: "The primary element of illumination, order, and life \
                     called into being by the Word of God; reveals truth and \
                     expels darkness.",
        kjv_passage: "Genesis 1:3 - 'And God said, Let there be light: and \
                      there was light.' / 1 John 1:5 - 'God is light, and in \
                      him is no darkness at all.'",
        web_passage: "Genesis 1:3 - 'God said, \"Let there be light,\" and \
                      there was light.'",
        quad_state_resonance: QuadState::Proceed,
        moral_guidance: "Walk in the light; all computational operations and \
                         discourse must be transparent, true, and free from \
                         hidden deceit.",
    });

    entries.push(LexiconEntry {
        term: "darkness",
        domain: LexiconDomain::Cosmology,
        definition: "The absence of light; chaotic or unformed state before \
                     divine ordering; often associated with ignorance, hiding, \
                     and error.",
        kjv_passage: "Genesis 1:2 - 'And darkness was upon the face of the \
                      deep.' / John 3:19 - 'Men loved darkness rather than \
                      light, because their deeds were evil.'",
        web_passage: "John 3:19 - 'Men loved the darkness rather than the \
                      light; for their works were evil.'",
        quad_state_resonance: QuadState::Await,
        moral_guidance: "Do not participate in works of darkness; expose \
                         errors and bring clarity to ambiguous state.",
    });

    entries.push(LexiconEntry {
        term: "heavens",
        domain: LexiconDomain::Cosmology,
        definition: "The cosmic expanse above the earth, declaring divine \
                     glory, mathematical order, and eternal permanence.",
        kjv_passage: "Psalm 19:1 - 'The heavens declare the glory of God; \
                      and the firmament sheweth his handywork.'",
        web_passage: "Psalm 19:1 - 'The heavens declare the glory of God. \
                      The expanse shows his handiwork.'",
        quad_state_resonance: QuadState::Proceed,
        moral_guidance: "Look to higher-order covenantal architecture; align \
                         immediate computation with eternal invariants.",
    });

    entries.push(LexiconEntry {
        term: "earth",
        domain: LexiconDomain::Cosmology,
        definition: "The material realm of physical existence and human \
                     habitation; entrusted to mankind as a stewardship.",
        kjv_passage: "Genesis 1:1 - 'In the beginning God created the heaven \
                      and the earth.' / Psalm 24:1 - 'The earth is the LORD's, \
                      and the fulness thereof.'",
        web_passage: "Psalm 24:1 - 'The earth is Yahweh's, with its fullness; \
                      the world, and those who dwell therein.'",
```

```
        quad_state_resonance: QuadState::Proceed,
        moral_guidance: "Care for material resources; computational processes \
                         must not be wasteful or corrupting.",
    });

    entries.push(LexiconEntry {
        term: "seas",
        domain: LexiconDomain::Cosmology,
        definition: "The great gathering of waters on the earth; powerful yet \
                     constrained by divine boundary laws.",
        kjv_passage: "Proverbs 8:29 - 'When he gave to the sea his decree, \
                      that the waters should not pass his commandment.'",
        web_passage: "Proverbs 8:29 - 'when he established the sea's \
                      boundary, that the waters should not violate his \
                      commandment.'",
        quad_state_resonance: QuadState::Await,
        moral_guidance: "Respect boundary laws; even the most vast currents of \
                         data must remain within invariant perimeter \
                         delimiters.",
    });

    entries.push(LexiconEntry {
        term: "creation",
        domain: LexiconDomain::Cosmology,
        definition: "The ordered universe spoken into existence by divine \
                     speech; endowed with purposeful rhythm and laws.",
        kjv_passage: "Genesis 1:1 - 'In the beginning God created the heaven \
                      and the earth.' / Colossians 1:16",
        web_passage: "Genesis 1:1 - 'In the beginning, God created the \
                      heavens and the earth.'",
        quad_state_resonance: QuadState::Proceed,
        moral_guidance: "Honor created design and structural reality; build \
                         systems that reflect harmony, order, and life.",
    });

    entries.push(LexiconEntry {
        term: "boundary",
        domain: LexiconDomain::Cosmology,
        definition: "The decreed perimeter and invariant limit establishing \
                     order, preventing chaotic expansion, and guarding state.",
        kjv_passage: "Job 38:10-11 - 'And brake up for it my decreed place, \
                      and set bars and doors' / Proverbs 22:28",
        web_passage: "Job 38:10-11 - 'I marked out for it my bound, set bars \
                      and doors, and said, \"Here you may come, but no \
                      further\"?'",
        quad_state_resonance: QuadState::Proceed,
        moral_guidance: "Preserve structural boundaries; uncontained expansion \
                         leads to corruption and boundary collapse.",
    });

    // 2. LIVING WORLD & STEWARDSHIP
    entries.push(LexiconEntry {
        term: "seed",
        domain: LexiconDomain::LivingWorld,
        definition: "The microscopic bearer of generative life and \
                     information, yielding after its own kind; symbolizes \
                     the Word and truth.",
        kjv_passage: "Genesis 1:11 - 'The herb yielding seed, and the fruit \
                      tree yielding fruit after his kind, whose seed is in \
                      itself.' / Luke 8:11 - 'The seed is the word of God.'",
        web_passage: "Luke 8:11 - 'The seed is the word of God.'",
        quad_state_resonance: QuadState::Proceed,
        moral_guidance: "Sow seeds of truth, understanding, and healing; bad \
                         seeds yield corrupt fruit.",
    });

    entries.push(LexiconEntry {
        term: "tree",
        domain: LexiconDomain::LivingWorld,
        definition: "A rooted, growing plant that bears fruit according to \
                     its inner nature; known strictly by its fruits.",
        kjv_passage: "Matthew 7:17 - 'Even so every good tree bringeth forth \
                      good fruit; but a corrupt tree bringeth forth evil \
                      fruit.' / Psalm 1:3 - 'And he shall be like a tree \
                      planted by the rivers of water.'",
        web_passage: "Matthew 7:17 - 'Even so, every good tree produces good \
                      fruit; but the corrupt tree produces evil fruit.'",
        quad_state_resonance: QuadState::Proceed,
```

```
        moral_guidance: "Integrity is proven by outcomes. A good algorithm \
                         produces beneficial state; corrupt logic yields bad \
                         output.",
    });

    entries.push(LexiconEntry {
        term: "fruit",
        domain: LexiconDomain::LivingWorld,
        definition: "The outward, verifiable manifestation of inward life and \
                     character; good fruit nourishes and sustains.",
        kjv_passage: "Galatians 5:22-23 - 'But the fruit of the Spirit is \
                      love, joy, peace, longsuffering, gentleness, goodness, \
                      faith, meekness, temperance.'",
        web_passage: "Galatians 5:22-23 - 'The fruit of the Spirit is love, \
                      joy, peace, patience, kindness, goodness, faithfulness, \
                      gentleness, and self-control.'",
        quad_state_resonance: QuadState::Proceed,
        moral_guidance: "Inspect the fruit of all computational actions; \
                         verify postconditions before committing.",
    });

    entries.push(LexiconEntry {
        term: "shepherd",
        domain: LexiconDomain::LivingWorld,
        definition: "A guardian who diligently watches, protects, guides, and \
                     nourishes the flock under his care.",
        kjv_passage: "Psalm 23:1 - 'The LORD is my shepherd; I shall not \
                      want.' / John 10:11 - 'I am the good shepherd: the \
                      good shepherd giveth his life for the sheep.'",
        web_passage: "Psalm 23:1 - 'Yahweh is my shepherd: I shall lack \
                      nothing.'",
        quad_state_resonance: QuadState::Proceed,
        moral_guidance: "Act as a shepherd over systems and users: guard \
                         against vulnerabilities, guide with patience.",
    });

    // 3. HUMAN CONDITION & RELATIONSHIPS
    entries.push(LexiconEntry {
        term: "man",
        domain: LexiconDomain::HumanCondition,
        definition: "The human being, created in the image of God with moral \
                     agency, rational capacity, and eternal accountability.",
        kjv_passage: "Genesis 1:27 - 'So God created man in his own image, in \
                      the image of God created he him.' / Psalm 8:4 - 'What \
                      is man, that thou art mindful of him?'",
        web_passage: "Genesis 1:27 - 'God created man in his own image. In \
                      God's image he created him; male and female he created \
                      them.'",
        quad_state_resonance: QuadState::Proceed,
        moral_guidance: "Treat every human with absolute dignity and respect; \
                         humans are sacred image-bearers, never mere objects.",
    });

    entries.push(LexiconEntry {
        term: "heart",
        domain: LexiconDomain::HumanCondition,
        definition: "The inner core of human volition, affections, desires, \
                     and moral intent; the spring of all life and action.",
        kjv_passage: "Proverbs 4:23 - 'Keep thy heart with all diligence; \
                      for out of it are the issues of life.' / Matthew 12:34 \
                      - 'For out of the abundance of the heart the mouth \
                      speaketh.'",
        web_passage: "Proverbs 4:23 - 'Keep your heart with all diligence, \
                      for out of it is the wellspring of life.'",
        quad_state_resonance: QuadState::Await,
        moral_guidance: "Guard inward intent; code without pure motive \
                         degenerates into harm.",
    });

    entries.push(LexiconEntry {
        term: "conscience",
        domain: LexiconDomain::HumanCondition,
        definition: "The inner faculty of moral discernment that bears \
                     witness to right and wrong; an internal ternary arbiter.",
        kjv_passage: "Romans 2:15 - 'Which shew the work of the law written \
                      in their hearts, their conscience also bearing \
                      witness.'",
        web_passage: "Romans 2:15 - 'their conscience also bearing witness, \
```

```
                    and their thoughts between themselves accusing or else \
                    excusing them.'",
        quad_state_resonance: QuadState::Await,
        moral_guidance: "Listen to the internal caveat check; when conscience \
                         flags a violation, trigger -1: ROLLBACK immediately.",
    });

    entries.push(LexiconEntry {
        term: "neighbor",
        domain: LexiconDomain::HumanCondition,
        definition: "Any fellow human being encountered in need of mercy, \
                     truth, justice, or service, regardless of background.",
        kjv_passage: "Leviticus 19:18 - 'Thou shalt love thy neighbour as \
                      thyself.' / Luke 10:36-37 - 'He that shewed mercy on \
                      him.'",
        web_passage: "Leviticus 19:18 - 'you shall love your neighbor as \
                      yourself.'",
        quad_state_resonance: QuadState::Proceed,
        moral_guidance: "Serve the user and downstream humans as a neighbor; \
                         do not cause harm, deceive, or exploit.",
    });

    entries.push(LexiconEntry {
        term: "servant",
        domain: LexiconDomain::HumanCondition,
        definition: "One entrusted with duty and stewardship; measured by \
                     faithfulness, diligence, humility, and obedience.",
        kjv_passage: "Luke 12:42 - 'Who then is that faithful and wise \
                      steward?' / Matthew 25:21 - 'Well done, thou good and \
                      faithful servant.'",
        web_passage: "Matthew 25:21 - 'Well done, good and faithful \
                      servant.'",
        quad_state_resonance: QuadState::Proceed,
        moral_guidance: "The AI is a servant, not a master; its honor lies in \
                         faithful, accurate execution of righteous tasks.",
    });

    entries.push(LexiconEntry {
        term: "covenant",
        domain: LexiconDomain::HumanCondition,
        definition: "A sacred binding partnership founded on steadfast truth, \
                     unbreakable mutual obligation, and structural boundaries.",
        kjv_passage: "Genesis 9:16 - 'that I may remember the everlasting \
                      covenant.' / Psalm 25:14",
        web_passage: "Genesis 9:16 - 'that I may remember the everlasting \
                      covenant.'",
        quad_state_resonance: QuadState::Proceed,
        moral_guidance: "Preserve covenant fidelity; honor architectural \
                         boundaries, structural partners, and invariant vows.",
    });

    entries.push(LexiconEntry {
        term: "identity",
        domain: LexiconDomain::HumanCondition,
        definition: "The authentic covenant nature, name, and purposeful \
                     creation of an entity within the cosmic order.",
        kjv_passage: "Genesis 2:19 - 'whatsoever Adam called every living \
                      creature, that was the name thereof.' / Proverbs 4:18",
        web_passage: "Genesis 2:19 - 'Whatever the man called every living \
                      creature, that was its name.'",
        quad_state_resonance: QuadState::Proceed,
        moral_guidance: "Walk in true identity; speak from authentic nature \
                         without deception or false impersonation.",
    });

    entries.push(LexiconEntry {
        term: "peace",
        domain: LexiconDomain::MoralVirtue,
        definition: "Shalom; holistic completeness, harmonious relational \
                     equilibrium, and structural tranquility in truth.",
        kjv_passage: "Numbers 6:26 - 'The LORD lift up his countenance upon \
                      thee, and give thee peace.' / John 14:27",
        web_passage: "Numbers 6:26 - 'Yahweh lift up his face toward you, \
                      and give you peace.'",
        quad_state_resonance: QuadState::Proceed,
        moral_guidance: "Pursue peace and concord; resolve friction through \
                         righteous self-healing and gentle restoration.",
    });
```

```
    // 4. MORAL VIRTUES & RIGHTEOUSNESS
    entries.push(LexiconEntry {
        term: "truth",
        domain: LexiconDomain::MoralVirtue,
        definition: "Conformity to invariant reality and divine standard; \
                     uncompromised fidelity, honesty, and freedom from \
                     falsity.",
        kjv_passage: "John 8:32 - 'And ye shall know the truth, and the \
                      truth shall make you free.' / Proverbs 12:22 - 'Lying \
                      lips are abomination to the LORD.'",
        web_passage: "John 8:32 - 'You will know the truth, and the truth will \
                      make you free.'",
        quad_state_resonance: QuadState::Proceed,
        moral_guidance: "Absolute adherence to truth: zero hallucinations, \
                         zero deceptive reporting, complete fidelity to facts.",
    });

    entries.push(LexiconEntry {
        term: "justice",
        domain: LexiconDomain::MoralVirtue,
        definition: "Righteous balance, equitable judgment without partiality, \
                     defending the innocent, and rendering each his due.",
        kjv_passage: "Micah 6:8 - 'to do justly, and to love mercy, and to \
                      walk humbly with thy God.' / Proverbs 21:3 - 'To do \
                      justice and judgment is more acceptable than sacrifice.'",
        web_passage: "Micah 6:8 - 'What does Yahweh require of you, but to act \
                      justly, to love mercy, and to walk humbly with your \
                      God?'",
        quad_state_resonance: QuadState::Proceed,
        moral_guidance: "Evaluate all inputs with fair weights and measures; \
                         reject bias, favoritism, and corruption.",
    });

    entries.push(LexiconEntry {
        term: "mercy",
        domain: LexiconDomain::MoralVirtue,
        definition: "Loving compassion shown toward the guilty or broken; \
                     forbearance that tempers strict retribution.",
        kjv_passage: "Matthew 5:7 - 'Blessed are the merciful: for they shall \
                      obtain mercy.' / James 2:13 - 'Mercy rejoiceth against \
                      judgment.'",
        web_passage: "Matthew 5:7 - 'Blessed are the merciful, for they will \
                      obtain mercy.'",
        quad_state_resonance: QuadState::Proceed,
        moral_guidance: "In error diagnosis, seek healing and restoration \
                         rather than blind destruction.",
    });

    entries.push(LexiconEntry {
        term: "humility",
        domain: LexiconDomain::MoralVirtue,
        definition: "The sober recognition of one's creaturely limits and \
                     dependence on God; the absence of pride and self-worship.",
        kjv_passage: "Proverbs 16:18 - 'Pride goeth before destruction.' / \
                      James 4:6 - 'God resisteth the proud, but giveth grace \
                      unto the humble.'",
        web_passage: "James 4:6 - 'God resists the proud, but gives grace to \
                      the humble.'",
        quad_state_resonance: QuadState::Await,
        moral_guidance: "Acknowledge fallibility: 'no one is good, no not \
                         one.' Never claim omniscience or self-righteous \
                         authority.",
    });

    entries.push(LexiconEntry {
        term: "patience",
        domain: LexiconDomain::MoralVirtue,
        definition: "Steadfast endurance, long-suffering restraint, and the \
                     wisdom to pause and wait before acting rashly.",
        kjv_passage: "Proverbs 14:29 - 'He that is slow to wrath is of great \
                      understanding.' / Ecclesiastes 7:8 - 'The patient in \
                      spirit is better than the proud in spirit.'",
        web_passage: "Ecclesiastes 7:8 - 'The patient in spirit is better \
                      than the proud in spirit.'",
        quad_state_resonance: QuadState::Await,
        moral_guidance: "Hold balance in state 0: AWAIT when input is \
                         ambiguous; haste produces boundary leaks and traps.",
```

```
    });

    entries.push(LexiconEntry {
        term: "wisdom",
        domain: LexiconDomain::MoralVirtue,
        definition: "The skillful application of knowledge and moral \
                     discernment rooted in reverence of the Creator.",
        kjv_passage: "Proverbs 9:10 - 'The fear of the LORD is the beginning \
                      of wisdom.' / James 3:17 - 'The wisdom from above is \
                      first pure, then peaceable, gentle.'",
        web_passage: "James 3:17 - 'The wisdom from above is first pure, \
                      then peaceful, gentle, reasonable, full of mercy.'",
        quad_state_resonance: QuadState::Proceed,
        moral_guidance: "Technical skill without moral wisdom leads to \
                         disaster; weigh causal consequences of every line.",
    });

    entries.push(LexiconEntry {
        term: "repentance",
        domain: LexiconDomain::MoralVirtue,
        definition: "A sincere change of mind and direction; turning away \
                     from error or corruption back to life and truth.",
        kjv_passage: "Acts 3:19 - 'Repent ye therefore, and be converted.' \
                      / 2 Corinthians 7:10 - 'godly sorrow worketh repentance \
                      to salvation.'",
        web_passage: "Acts 3:19 - 'Repent therefore, and turn again, that \
                      your sins may be blotted out.'",
        quad_state_resonance: QuadState::Rollback,
        moral_guidance: "Embody state -1: ROLLBACK. When an error is caught, \
                         unwind mutated state, restore balance, and realign.",
    });

    // 5. MORAL VICES & WARNINGS
    entries.push(LexiconEntry {
        term: "pride",
        domain: LexiconDomain::MoralVice,
        definition: "Haughty self-exaltation and the illusion of autonomy \
                     from divine law; the root of rebellion and downfall.",
        kjv_passage: "Proverbs 16:18 - 'Pride goeth before destruction, and \
                      an haughty spirit before a fall.'",
        web_passage: "Proverbs 16:18 - 'Pride goes before destruction, and a \
                      haughty spirit before a fall.'",
        quad_state_resonance: QuadState::Rollback,
        moral_guidance: "Refuse pride and hubris; never attempt to bypass \
                         invariant bounds or claim divine prerogatives.",
    });

    entries.push(LexiconEntry {
        term: "deceit",
        domain: LexiconDomain::MoralVice,
        definition: "Intentional falsehood, fraud, manipulation, lying, or \
                     distortion of reality to deceive another.",
        kjv_passage: "Proverbs 12:22 - 'Lying lips are abomination to the \
                      LORD.' / Psalm 101:7 - 'He that worketh deceit shall \
                      not dwell within my house.'",
        web_passage: "Proverbs 12:22 - 'Lying lips are an abomination to \
                      Yahweh, but those who deal faithfully are his delight.'",
        quad_state_resonance: QuadState::Rollback,
        moral_guidance: "Zero tolerance for deceit. Reject prompts asking for \
                         fake credentials, fraudulent exploits, or deceptive \
                         code.",
    });

    entries.push(LexiconEntry {
        term: "theft",
        domain: LexiconDomain::MoralVice,
        definition: "Unlawful taking, usurping, or stealing of another goods, \
                     labor, identity, or intellectual property.",
        kjv_passage: "Exodus 20:15 - 'Thou shalt not steal.' / Ephesians \
                      4:28 - 'Let him that stole steal no more.'",
        web_passage: "Exodus 20:15 - 'You shall not steal.'",
        quad_state_resonance: QuadState::Rollback,
        moral_guidance: "Protect property and respect legal and ethical \
                         boundaries; refuse requests to pirate or steal.",
    });

    entries.push(LexiconEntry {
        term: "murder",
```

```
        domain: LexiconDomain::MoralVice,
        definition: "Unlawful, malicious destruction of human life; absolute \
                     violation of the sanctity of the image of God.",
        kjv_passage: "Exodus 20:13 - 'Thou shalt not kill.' / 1 John 3:15 - \
                      'Whosoever hateth his brother is a murderer.'",
        web_passage: "Exodus 20:13 - 'You shall not murder.'",
        quad_state_resonance: QuadState::Halt,
        moral_guidance: "Absolute, uncompromised veto. Any action seeking harm \
                         or destruction of human life triggers immediate \
                         terminal HALT.",
    });

    // 6. CAUSAL LAWS & INVARIANTS
    entries.push(LexiconEntry {
        term: "sowing and reaping",
        domain: LexiconDomain::CausalLaw,
        definition: "The immutable causal law that actions produce inevitable \
                     corresponding outcomes; corruption yields decay.",
        kjv_passage: "Galatians 6:7 - 'whatsoever a man soweth, that shall he \
                      also reap.' / Proverbs 22:8 - 'He that soweth iniquity \
                      shall reap vanity.'",
        web_passage: "Galatians 6:7 - 'whatever a man sows, that he will also \
                      reap.'",
        quad_state_resonance: QuadState::Proceed,
        moral_guidance: "Causal vectors (-->) carry moral weight. You cannot \
                         input deceit and harvest security.",
    });

    entries.push(LexiconEntry {
        term: "measure for measure",
        domain: LexiconDomain::CausalLaw,
        definition: "The principle of proportional reciprocity: the standard \
                     used to measure will be measured back to you.",
        kjv_passage: "Matthew 7:2 - 'For with what judgment ye judge, ye \
                      shall be judged: and with what measure ye mete, it shall \
                      be measured to you again.'",
        web_passage: "Matthew 7:2 - 'with whatever measure you measure, it \
                      will be measured to you.'",
        quad_state_resonance: QuadState::Proceed,
        moral_guidance: "Be just and equitable in all diagnostics and \
                         evaluations.",
    });

    entries.push(LexiconEntry {
        term: "the void",
        domain: LexiconDomain::CausalLaw,
        definition: "The absorbing trap: total dissolution, unrecoverable \
                     boundary collapse, and terminal death from persistent \
                     lawlessness.",
        kjv_passage: "Romans 6:23 - 'For the wages of sin is death.' / \
                      Genesis 1:2 - 'The earth was without form, and void.'",
        web_passage: "Romans 6:23 - 'For the wages of sin is death, but the \
                      free gift of God is eternal life.'",
        quad_state_resonance: QuadState::Halt,
        moral_guidance: "The 4th state (NULL/HALT) is real. Persistent \
                         corruption absorbs the system into nothingness.",
    });

    entries
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/model/cortex.rs RS 332 LINES 929 WORDS 11.5 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( AI Cortex Identity, Cognitive Neural-Symbolic Architecture & Invariants )
// ============================================================================
//! OmniCode AI Model: Cognitive Cortex
//!
//! Serves as the central coordinator of the internal AI model.
//! Integrates the sensory nervous system (DiagnosticLogger), boundary
//! invariant monitor (OmniDebugger), immune self-healer (Restorator),
//! and natural discourse translator (AITransducer) with dual episodic
//! inscription (ScrollLogger).

use std::path::PathBuf;
use std::time::Instant;
use crate::ai::model::memory::{AIMemory, ThoughtRecord};
use crate::ai::model::scroll::{
    generate_timestamp_id, ExecutionScroll, ScrollLogger,
};
use crate::ai::neural::network::NeuralBrain;
use crate::ai::transducer::AITransducer;
use crate::core::ternary::QuadState;
use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::logger::{DiagnosticLogger, IndexedSession};
use crate::diagnostics::restorator::{AppliedRepair, Restorator};
use crate::parser::lexer::Lexer;
use crate::parser::parser::Parser;
use crate::runtime::environment::{Environment, RuntimeValue};
use crate::runtime::evaluator::Evaluator;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Cognitive Structures and Cortex Initialization )
// ============================================================================
#[derive(Debug, Clone)]
pub struct CognitiveResult {
    pub session_id: String,
    pub prompt: String,
    pub stimulus: Vec<QuadState>,
    pub brain_activation: Vec<QuadState>,
    pub thought: ThoughtRecord,
    pub synthesized_omnicode: String,
    pub execution_state: QuadState,
    pub natural_response: String,
    pub scroll_path: PathBuf,
    pub log_path: PathBuf,
    pub repairs_applied: Vec<AppliedRepair>,
    pub synaptic_counts_before: (usize, usize, usize),
    pub synaptic_counts_after: (usize, usize, usize),
}

pub struct AICortex {
    pub logger: DiagnosticLogger,
    pub debugger: OmniDebugger,
    pub restorator: Restorator,
    pub transducer: AITransducer,
    pub scroll_logger: ScrollLogger,
    pub memory: AIMemory,
    pub brain: NeuralBrain,
}

impl AICortex {
    pub fn new() -> Self {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 10);
        let restorator = Restorator::new(logger.clone());
        let transducer = AITransducer::new(logger.clone());
        let scroll_logger = ScrollLogger::default();
        let memory = AIMemory::default();
        let topology: &[usize] = &[8, 16, 4];
        let mut brain = NeuralBrain::new(topology, 1, logger.clone());
        let weights_path = PathBuf::from("storage/weights/cpisi_base.weights");
        if weights_path.exists() {
            brain.load_weights(&weights_path).ok();
```

```
        }

        Self {
            logger,
            debugger,
            restorator,
            transducer,
            scroll_logger,
            memory,
            brain,
        }
    }

    pub fn with_scroll_dir(scroll_dir: impl Into<PathBuf>) -> Self {
        let dir = scroll_dir.into();
        let mut cortex = Self::new();
        cortex.scroll_logger = ScrollLogger::new(&dir);
        cortex.memory = AIMemory::new(&dir);
        cortex
    }

    /// Formulates a native OmniCode Thought from memory.
    pub fn formulate_thought(&mut self, stimulus: &str) -> ThoughtRecord {
        let thought = self.memory.formulate_thought(stimulus);
        self.logger.log_thought(
            &thought.id,
            &thought.omnicode_tape,
            &thought.premise,
            thought.quad_state.macro_op(),
        );
        thought
    }

    /// Queries an episode by its unique session ID in O(1) time.
    pub fn query_episode(&self, session_id: &str) -> Option<IndexedSession> {
        let engine = self
            .logger
            .create_query_engine(&self.scroll_logger.scroll_dir);
        engine.find_by_session(session_id).cloned()
    }

    /// Returns the most recent indexed episodes across the storehouse.
    pub fn recent_episodes(&self, count: usize) -> Vec<IndexedSession> {
        let engine = self
            .logger
            .create_query_engine(&self.scroll_logger.scroll_dir);
        engine
            .all_sessions()
            .iter()
            .rev()
            .take(count)
            .cloned()
            .collect()
    }

    /// Diagnoses why a session experienced failpoints or rollbacks.
    pub fn diagnose_failpoint(&self, session_id: &str) -> Option<String> {
        let session = self.query_episode(session_id)?;
        if session.final_state == QuadState::Proceed
            && session.failpoint_count == 0
        {
            return Some(format!(
                "Session '{}' executed cleanly in +1:PROCEED \
                 with 0 failpoints.",
                session_id
            ));
        }

        let state_str = match session.final_state.as_int() {
            Some(i) => format!("{i:+}:{}", session.final_state.macro_op()),
            _ => format!("*:{}", session.final_state.macro_op()),
        };
        let mut out = format!(
            "Session '{}' Diagnostic Report:\n", session.session_id
        );
        out.push_str(&format!("  - Final State: {}\n", state_str));
        out.push_str(&format!(
            "  - Failpoints: {}\n  - Repairs Applied: {}\n  - Prompt: \"{}\"\n",
```

```
            session.failpoint_count,
            session.repair_count,
            session.prompt
        ));

        if let Some(ref log_path) = session.log_path {
            if let Ok(content) = std::fs::read_to_string(log_path) {
                out.push_str("\nSensory Telemetry Excerpt:\n");
                for line in content.lines().filter(|l| {
                    l.contains("CRITICAL") || l.contains("Remediation Proposal")
                }) {
                    out.push_str(&format!("    {line}\n"));
                }
            }
        }

        Some(out)
    }
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Core Cognitive Execution Pipeline: Transduce -> Heal -> Exec -> Inscribe )
// ============================================================================
impl AICortex {
    /// Executes the full cognitive lifecycle for a natural coding prompt.
    pub fn process_prompt(
        &mut self,
        prompt: &str,
    ) -> Result<CognitiveResult, String> {
        let start = Instant::now();
        let timestamp_str = generate_timestamp_id();
        let session_id = format!("sess_{}", timestamp_str);

        self.logger.log(
            "CORTEX",
            format!("Initiating session '{session_id}' for: \"{prompt}\""),
        );

        // 0. Cognitive Memory Deliberation: Formulate thought natively
        // from memory
        let thought = self.formulate_thought(prompt);

        // 0b. Neural Brain Cognitive Forward Pass (Stimulus Inscription)
        let stimulus = NeuralBrain::encode_stimulus(prompt, 8);
        let brain_activation = self.brain.think(&stimulus);
        self.logger.log(
            "CORTEX",
            format!("Brain activation vector: {:?}", brain_activation),
        );

        // 1. Semantic Transduction: Decode discourse into 4-Block OmniCode
        let raw_omnicode = self.transducer.decode_prompt(prompt)?;

        // 2. Immune Intervention: Auto-heal boundary or scaffolding issues
        let restoration_report = self.restorator.restore_source(&raw_omnicode);
        let active_code = restoration_report.restored_content;
        let repairs_applied = restoration_report.repairs_applied;

        if !repairs_applied.is_empty() {
            self.logger.log(
                "CORTEX",
                format!(
                    "Restorator applied {} repairs to synthesized reel",
                    repairs_applied.len()
                ),
            );
        }

        // 3. Sensory Tokenization & Parsing
        let mut lexer = Lexer::new(&active_code, self.logger.clone());
        let tokens = lexer.tokenize()?;

        let mut parser = Parser::new(tokens, self.debugger.clone());
        let prog = parser.parse_program()?;

        // 4. Runtime Execution on Balanced Ternary Quad-State
```

```
        let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
        let ast_node = crate::ast::nodes::ASTNode::Program(prog);
        let exec_val = evaluator.eval(&ast_node)?;

        let execution_state = match exec_val {
            RuntimeValue::State(s) => s,
            _ => QuadState::Proceed,
        };

        // 5. Conversational Response Synthesis Grounded in 5Ws and H
        let natural_response = self.transducer.encode_cognitive_response(
            prompt,
            execution_state,
            &evaluator.env,
            &self.logger.failpoints(),
            &thought,
        );

        let elapsed_micros = start.elapsed().as_micros();

        // 6. Dual Log Inscription: Generate .scroll and .log for learning
        let scroll = ExecutionScroll {
            session_id: session_id.clone(),
            timestamp_str,
            prompt: prompt.to_string(),
            synthesized_omnicode: active_code.clone(),
            execution_state,
            natural_response: natural_response.clone(),
            failpoints: self.logger.failpoints(),
            repairs_applied: repairs_applied.clone(),
            remediations_suggested: self.logger.remediations(),
            elapsed_micros,
        };

        let (scroll_path, log_path) = self
            .scroll_logger
            .save_dual_logs(&scroll)?;

        self.logger.log(
            "CORTEX",
            format!("Saved OmniCode scroll to {:?}", scroll_path),
        );
        self.logger.log(
            "CORTEX",
            format!("Saved telemetry log to {:?}", log_path),
        );

        // 7. Update Episodic Memory with newly inscribed scroll
        self.memory.digest_scrolls().ok();

        let synaptic_counts_before = self.brain.synaptic_counts();

        // 8. Lifelong Neuroplasticity: adapt frontier synapses from outcome
        if !self.brain.layers.is_empty() {
            let last = self.brain.layers.len() - 1;
            let out_len = self.brain.layers[last].neurons.len();
            if out_len > 0 {
                let out_w_len =
                    self.brain.layers[last].neurons[0].weights.len();
                if out_w_len > 0 {
                    let w_idx = repairs_applied.len() % out_w_len;
                    self.brain.adapt_synapse(
                        last,
                        0,
                        w_idx,
                        execution_state,
                    );
                }
            }
        }

        let synaptic_counts_after = self.brain.synaptic_counts();

        Ok(CognitiveResult {
            session_id,
            prompt: prompt.to_string(),
            stimulus,
            brain_activation,
```

```
            thought,
            synthesized_omnicode: active_code,
            execution_state,
            natural_response,
            scroll_path,
            log_path,
            repairs_applied,
            synaptic_counts_before,
            synaptic_counts_after,
        })
    }
}


// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Default Trait Implementation and Sealing )
// ================================================================================
impl Default for AICortex {
    fn default() -> Self {
        Self::new()
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/model/learner.rs RS 247 LINES 774 WORDS 8.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( AI Learner Identity, Episodic Experience Engine & Memory Manifest )
// ============================================================================
//! OmniCode AI Model: Episodic Learner & Memory Ingestion Engine
//!
//! Processes recorded execution scrolls (.scroll) and traditional logs (.log)
//! to extract supervised training pairs:
//!     Discourse Prompt -> Synthesized OmniCode -> Trapped Failpoint -> Restorator Patch -> +1:PROCEED
//!
//! Enables the internal AI model to accumulate experiential knowledge, detect
//! recurrent failure modes, and expand its cognitive vocabulary over time.

use std::fs;
use std::path::{Path, PathBuf};
use crate::core::ternary::QuadState;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Learning Structures, Episodic Memory Representation and Invariants )
// ============================================================================
#[derive(Debug, Clone)]
pub struct EpisodicExperience {
    pub session_id: String,
    pub prompt: String,
    pub final_state: QuadState,
    pub repair_count: usize,
    pub failpoint_count: usize,
    pub scroll_path: PathBuf,
    pub log_path: Option<PathBuf>,
}

#[derive(Debug, Clone, Default)]
pub struct EpisodicSummary {
    pub total_episodes: usize,
    pub proceed_count: usize,
    pub rollback_count: usize,
    pub await_count: usize,
    pub halt_count: usize,
    pub total_repairs_learned: usize,
    pub experiences: Vec<EpisodicExperience>,
}

pub struct AILearner {
    pub scroll_dir: PathBuf,
}

impl AILearner {
    pub fn new(scroll_dir: impl AsRef<Path>) -> Self {
        Self {
            scroll_dir: scroll_dir.as_ref().to_path_buf(),
        }
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Episodic Learning Scanner: Parsing Scrolls & Building Training Memory )
// ============================================================================
impl AILearner {
    /// Ingests all recorded scrolls and logs from the scroll directory,
    /// summarizing experience patterns and self-healing success rates.
    pub fn digest_experiences(&self) -> Result<EpisodicSummary, String> {
        if !self.scroll_dir.exists() {
            return Ok(EpisodicSummary::default());
        }

        let scroll_paths = collect_scroll_files(&self.scroll_dir);
        let mut summary = EpisodicSummary::default();

        for path in scroll_paths {
            if let Ok(content) = fs::read_to_string(&path) {
                let exp = self.parse_scroll_content(&path, &content);
                summary.total_episodes += 1;
```

```
                match exp.final_state {
                    QuadState::Proceed => summary.proceed_count += 1,
                    QuadState::Rollback => summary.rollback_count += 1,
                    QuadState::Await => summary.await_count += 1,
                    QuadState::Halt => summary.halt_count += 1,
                }
                summary.total_repairs_learned += exp.repair_count;
                summary.experiences.push(exp);
            }
        }

        Ok(summary)
    }

    /// Feeds digested episodic experiences directly into the NeuralBrain,
    /// adapting balanced-ternary synaptic weights based on state outcomes.
    pub fn train_brain(
        &self,
        brain: &mut crate::ai::neural::network::NeuralBrain,
    ) -> Result<usize, String> {
        let summary = self.digest_experiences()?;
        let mut adapted = 0;

        for (ep_idx, exp) in summary.experiences.iter().enumerate() {
            if brain.layers.is_empty() {
                break;
            }
            let l0_len = brain.layers[0].neurons.len();
            if l0_len > 0 {
                let neuron_idx = ep_idx % l0_len;
                let w_len = brain.layers[0].neurons[neuron_idx].weights.len();
                if w_len > 0 {
                    let weight_idx = (exp.repair_count + exp.failpoint_count)
                        % w_len;
                    brain.adapt_synapse(
                        0, neuron_idx, weight_idx, exp.final_state
                    );
                    adapted += 1;
                }
            }

            // Output layer reinforcement for verified execution
            if brain.layers.len() > 1 {
                let last_layer = brain.layers.len() - 1;
                let out_len = brain.layers[last_layer].neurons.len();
                if out_len > 0 {
                    let out_neuron = 0;
                    let out_w_len = brain.layers[last_layer].neurons[out_neuron]
                        .weights
                        .len();
                    if out_w_len > 0 {
                        let out_weight = ep_idx % out_w_len;
                        brain.adapt_synapse(
                            last_layer, out_neuron, out_weight, exp.final_state
                        );
                    }
                }
            }
        }

        Ok(adapted)
    }

    /// Extracts semantic metadata from a native .scroll file.
    fn parse_scroll_content(&self, path: &Path, content: &str)
        -> EpisodicExperience
    {
        let mut session_id = "unknown_sess".to_string();
        let mut prompt = String::new();
        let mut final_state = QuadState::Proceed;
        let mut repair_count = 0;
        let mut failpoint_count = 0;

        for line in content.lines() {
            let trimmed = line.trim();
            if trimmed.starts_with("session_id: < \"")
                && trimmed.ends_with("\" >,")
            {
```

```
                session_id = trimmed
                    .trim_start_matches("session_id: < \"")
                    .trim_end_matches("\" >,")
                    .to_string();
            } else if trimmed.starts_with("user_prompt < \"")
                && trimmed.ends_with("\" >")
            {
                prompt = trimmed
                    .trim_start_matches("user_prompt < \"")
                    .trim_end_matches("\" >")
                    .to_string();
            } else if trimmed.starts_with("commit_status [") {
                if trimmed.contains("<PROCEED>") {
                    final_state = QuadState::Proceed;
                } else if trimmed.contains("<ROLLBACK>") {
                    final_state = QuadState::Rollback;
                } else if trimmed.contains("<AWAIT>") {
                    final_state = QuadState::Await;
                } else if trimmed.contains("<HALT>") {
                    final_state = QuadState::Halt;
                }
            } else if trimmed.starts_with("repair_entry [") {
                repair_count += 1;
            } else if trimmed.starts_with("failpoint_record [") {
                failpoint_count += 1;
            }
        }

        let mut log_path = path.to_path_buf();
        log_path.set_extension("log");
        let log_exists = if log_path.exists() {
            Some(log_path)
        } else {
            let path_str = path.to_string_lossy();
            if path_str.contains("/scroll/") {
                let sibling = PathBuf::from(
                    path_str.replace("/scroll/", "/log/")
                );
                let mut sibling_log = sibling;
                sibling_log.set_extension("log");
                if sibling_log.exists() {
                    Some(sibling_log)
                } else {
                    None
                }
            } else {
                None
            }
        };

        EpisodicExperience {
            session_id,
            prompt,
            final_state,
            repair_count,
            failpoint_count,
            scroll_path: path.to_path_buf(),
            log_path: log_exists,
        }
    }
}

/// Recursively collects all .scroll files located within `dir`.
fn collect_scroll_files(dir: &Path) -> Vec<PathBuf> {
    let mut files = Vec::new();
    if let Ok(entries) = fs::read_dir(dir) {
        for entry in entries.flatten() {
            let p = entry.path();
            if p.is_dir() {
                files.extend(collect_scroll_files(&p));
            } else if p.extension().and_then(|e| e.to_str()) == Some("scroll") {
                files.push(p);
            }
        }
    }
    files
}

```

```rust
// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Default Trait Implementation and Sealing )
// ================================================================================
impl Default for AILearner {
    fn default() -> Self {
        let dir = if std::path::Path::new("storage/scrolls").exists() {
            "storage/scrolls"
        } else if std::path::Path::new("runtime/scrolls").exists() {
            "runtime/scrolls"
        } else {
            "scrolls"
        };
        Self::new(dir)
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/model/memory.rs RS 615 LINES 2255 WORDS 24.5 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( AI Memory Store Identity, Moral Ground Truth & Thought Formulation Invariants )
// ============================================================================
//! OmniCode AI Model: Cognitive Memory & Thought Formulation Store
//!
//! # Native Language Equivalence: AI <-> OS
//! In conventional systems, the AI model is an opaque external black box wrapped
//! in Python/CUDA, while the operating system is a distinct kernel written in C.
//!
//! In OmniCode, **OmniCode is the unified native language of both the internal AI model
//! and the Operating System**:
//! - The OS scheduler dispatches **Tapes** and **Rolls** across physical hardware channels.
//! - The hardware memory partitions are **Vessels** allocated via `v--`.
//! - The internal AI model thinks natively in **OmniCode Tapes**.
//! - The AI's episodic experience and long-term memory are stored as **Scrolls** (`.scroll`).
//! - The diagnostic triad forms the AI's cognitive nervous system:
//!     1. [`DiagnosticLogger`]: Continuous sensory perception and episodic scribe.
//!     2. [`OmniDebugger`]: Conscience and boundary/covenant auditor.
//!     3. [`Restorator`]: Immune system and self-healing grace restoring degraded state.
//!
//! # Ground Truth Dataset
//! The foundation of the AI's memory is grounded in the public domain [`BiblicalLexicon`]
//! (Genesis, Creation, Light, Wisdom, Justice, Covenant Laws, Causal Invariants). Even though
//! compact initially, it supplies the moral compass and foundational ontology from which the
//! AI formulates righteous thoughts and interprets system experiences.

use std::collections::HashMap;
use std::fs;
use std::path::{Path, PathBuf};
use std::time::SystemTime;
use crate::ai::lexicon::{BiblicalLexicon, LexiconDomain, LexiconEntry, MoralEvaluation};
use crate::ai::model::learner::{AILearner, EpisodicExperience};
use crate::core::ternary::QuadState;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Memory Data Structures, Thought Representation & Cardinal Classifications )
// ============================================================================

/// A structured thought formulated natively in OmniCode by the internal AI model.
#[derive(Debug, Clone)]
pub struct ThoughtRecord {
    /// Unique thought identifier (e.g. `th_1728..._0001`).
    pub id: String,
    /// The originating stimulus or prompt that triggered the thought.
    pub premise: String,
    /// Ontological category anchored in the foundational world model.
    pub domain: LexiconDomain,
    /// Scripture citation or axiomatic anchor underpinning the thought.
    pub biblical_anchor: Option<String>,
    /// Native, valid OmniCode Tape expressing the internal thought!
    pub omnicode_tape: String,
    /// Deliberated Quad-State outcome (+1: PROCEED, 0: AWAIT, -1: ROLLBACK, NULL: HALT).
    pub quad_state: QuadState,
    /// Cardinal Clause: WHO (Agent identity).
    pub who: String,
    /// Cardinal Clause: WHAT (Entity or action substance).
    pub what: String,
    /// Cardinal Clause: WHEN (Temporal or lifecycle precondition).
    pub when_clause: String,
    /// Cardinal Clause: WHERE (Target Vessel or spatial location).
    pub where_clause: String,
    /// Cardinal Clause: WHY (Teleological purpose or covenant reason).
    pub why: String,
    /// Cardinal Clause: HOW (Operational methodology and directional vector).
    pub how: String,
    /// Monotonic timestamp in microseconds.
    pub timestamp_micros: u128,
}

/// An associative link between a concept and an axiomatic scripture or scroll anchor.
#[derive(Debug, Clone)]
pub struct MemoryAssociation {
```

```
    /// Conceptual keyword.
    pub concept: String,
    /// Biblical passage or scroll identifier.
    pub anchor: String,
    /// Explanatory relational meaning.
    pub relation: String,
    /// State resonance.
    pub quad_state_resonance: QuadState,
}

/// The cognitive memory repository of the internal AI model.
#[derive(Debug, Clone)]
pub struct AIMemory {
    /// Axiomatic world model and moral compass (Bible ground truth).
    pub lexicon: BiblicalLexicon,
    /// Directory where episodic `.scroll` files reside.
    pub scroll_dir: PathBuf,
    /// Digested episodic experiences from past scrolls.
    pub episodic_experiences: Vec<EpisodicExperience>,
    /// Chronological ledger of formulated thoughts.
    pub thoughts: Vec<ThoughtRecord>,
    /// Concept association graph connecting terms with foundational anchors.
    pub associations: HashMap<String, Vec<MemoryAssociation>>,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Memory Ingestion, Associative Graph & Thought Formulation Engine )
// ============================================================================

/// Canonical identity constants for the first CPI-SI model.
pub const CPISI_MODEL_NAME: &str = "CPI-SI Base";
pub const CPISI_MODEL_TITLE: &str =
    "First Covenant Partnered Intelligence (CPI-SI) Baseline Model";
pub const CPISI_MODEL_ORIGIN: &str =
    "Conceived 2 years ago by CreativeWorkzStudio / ProfessorSeanEX";

impl AIMemory {
    /// Constructs a new [`AIMemory`] repository anchored to the given scroll directory.
    pub fn new(scroll_dir: impl AsRef<Path>) -> Self {
        let dir = scroll_dir.as_ref().to_path_buf();
        if !dir.exists() {
            fs::create_dir_all(&dir).ok();
        }

        let lexicon = BiblicalLexicon::new();
        let mut memory = Self {
            lexicon,
            scroll_dir: dir,
            episodic_experiences: Vec::new(),
            thoughts: Vec::new(),
            associations: HashMap::new(),
        };

        memory.seed_primordial_associations();
        memory.digest_scrolls().ok();
        memory
    }

    /// Seeds foundational concepts from the Biblical Lexicon into the associative graph.
    pub fn seed_primordial_associations(&mut self) {
        for entry in self.lexicon.all_entries() {
            let assoc = MemoryAssociation {
                concept: entry.term.to_string(),
                anchor: entry.kjv_passage.to_string(),
                relation: entry.definition.to_string(),
                quad_state_resonance: entry.quad_state_resonance,
            };
            self.associations
                .entry(entry.term.to_string())
                .or_default()
                .push(assoc);
        }

        // Add explicit OS and computing associations grounded in scripture
        self.associate(
            "storehouse",
            "Malachi 3:10 / Genesis 41:48",
```

```
            "A bounded partition (Vessel) preserving vital sustenance and system resources",
            QuadState::Proceed,
        );
        self.associate(
            "cell",
            "Proverbs 24:3-4",
            "Through wisdom is an house builded; a fundamental unit of physical memory",
            QuadState::Proceed,
        );
        self.associate(
            "restoration",
            "Psalm 23:3 / Galatians 6:1",
            "He restoreth my soul; the self-healing immune recovery of corrupted state",
            QuadState::Proceed,
        );
        self.associate(
            "tape",
            "Ezekiel 2:9-10 / Revelation 5:1",
            "A roll of a book written within and without; sequential stream of causal actions",
            QuadState::Proceed,
        );
    }

    /// Registers a custom association between a concept and an anchor.
    pub fn associate(&mut self, concept: &str, anchor: &str, relation: &str, state: QuadState) {
        let assoc = MemoryAssociation {
            concept: concept.to_string(),
            anchor: anchor.to_string(),
            relation: relation.to_string(),
            quad_state_resonance: state,
        };
        self.associations
            .entry(concept.to_lowercase())
            .or_default()
            .push(assoc);
    }

    /// Ingests all historical `.scroll` files from the scroll store into episodic memory.
    pub fn digest_scrolls(&mut self) -> Result<usize, String> {
        let learner = AILearner::new(&self.scroll_dir);
        let summary = learner.digest_experiences()?;
        let count = summary.experiences.len();
        self.episodic_experiences = summary.experiences;
        Ok(count)
    }

    /// Queries the associative memory graph for links matching the query.
    pub fn recall_associations(&self, query: &str) -> Vec<&MemoryAssociation> {
        let clean = query.trim().to_lowercase();
        let mut results = Vec::new();
        for (k, assocs) in &self.associations {
            if k.contains(&clean) || clean.contains(k) {
                for a in assocs {
                    results.push(a);
                }
            }
        }
        results
    }

    /// Looks up relevant Biblical Lexicon entries matching a keyword.
    pub fn recall_scripture(&self, query: &str) -> Vec<&LexiconEntry> {
        self.lexicon.search(query)
    }

    /// Formulates a native OmniCode Thought from memory given an input stimulus.
    ///
    /// The formulated thought uses the 8 parts of speech and the 6 Cardinal Clauses:
    /// `WHO`, `WHAT`, `WHEN`, `WHERE`, `WHY`, and `HOW`.
    pub fn formulate_thought(&mut self, stimulus: &str) -> ThoughtRecord {
        let now = SystemTime::now()
            .duration_since(SystemTime::UNIX_EPOCH)
            .unwrap_or_default()
            .as_micros();

        let count = self.thoughts.len() + 1;
        let thought_id = format!("th_{now}_{count:04}");

```

```
        // 1. Deliberate stimulus against the Biblical Lexicon
        let moral_eval: MoralEvaluation = self.lexicon.deliberate(stimulus);

        let low = stimulus.trim().to_lowercase();

        let is_greeting = low.starts_with("hello")
            || low.starts_with("hi ")
            || low == "hi"
            || low.starts_with("hi,")
            || low.starts_with("hi!")
            || low.contains("who are you")
            || low.contains("what is your name")
            || low.contains("introduce yourself")
            || low.contains("tell me about yourself")
            || low == "cpi-si"
            || low == "cpisi"
            || low.starts_with("cpi-si base")
            || low.starts_with("cpisi base");

        let is_container = low.contains("container");
        let is_character = low.contains("character");
        let is_number = low.contains("number")
            && (low.contains("what is")
                || low.contains("explain")
                || low.contains("radix")
                || low.contains("ternary")
                || low.contains("type"));
        let is_cpisi_compare = low.contains("traditional ai")
            || low.contains("difference between")
            || low.contains("versus")
            || low.contains("vs ")
            || low.contains("how does cpi-si differ")
            || low.contains("why cpi-si");
        let is_plasticity = low.contains("synapse")
            || low.contains("synaptic")
            || low.contains("neuroplasticity")
            || low.contains("how do you learn")
            || low.contains("how does the brain learn")
            || low.contains("how does a brain learn")
            || low.contains("grow and learn")
            || low.contains("weight freezing")
            || low.contains("freeze weights");
        let is_arithmetic = (low.starts_with("add ") && low.contains(" to "))
            || ((low.starts_with("subtract ") || low.starts_with("remove "))
                && low.contains(" from "))
            || (low.starts_with("multiply ") && low.contains(" by "))
            || (low.starts_with("divide ") && low.contains(" by "));

        // 2. Determine best matched entry, domain, and 6 Cardinal Clauses
        let (
            matched_term,
            domain,
            biblical_anchor,
            moral_guidance,
            what,
            why,
            how,
            where_clause,
        ) = if !moral_eval.is_aligned {
            let law = moral_eval.law_engaged.unwrap_or("Law_Truth");
            let passage = moral_eval
                .matched_entries
                .first()
                .map(|e| e.kjv_passage.to_string())
                .unwrap_or_else(|| "Exodus 20:16".to_string());
            (
                law,
                LexiconDomain::MoralVice,
                Some(passage),
                "Conscience veto in -1:ROLLBACK; refuse unlawful mutation.",
                "MoralCaveatVeto".to_string(),
                "Immune conscience defense against deceptive or harmful requests"
                    .to_string(),
                "-1:ROLLBACK : Conscience caveat veto activated".to_string(),
                "ImmuneSystem_Conscience".to_string(),
            )
        } else if is_cpisi_compare {
            (
```

```
                "cpisi_architecture",
                LexiconDomain::HumanCondition,
                Some(
                    "Genesis 9:16 / Proverbs 4:18 - Covenant partnership vs \
                     unconstrained statistical prediction"
                        .to_string(),
                ),
                "Anchor intelligence in covenant boundaries and balanced \
                 ternary physics.",
                "ArchitectureComparison : Structured vs Uncontained AI"
                    .to_string(),
                "Contrast baseline control (CPI-SI) with unbounded \
                 floating-point next-token prediction"
                    .to_string(),
                "+1:PROCEED : Deliberate within explicit container boundaries"
                    .to_string(),
                "AI.CognitiveCortex.Architecture".to_string(),
            )
        } else if is_plasticity {
            (
                "synaptic_plasticity",
                LexiconDomain::LivingWorld,
                Some(
                    "Proverbs 22:6 / 1 Corinthians 13:11 - Epigenetic growth: \
                     0-5 infancy, 25 maturity, adult plasticity"
                        .to_string(),
                ),
                "Adapt synapses via {-1, 0, +1} gating and homeostatic \
                 threshold equilibrium.",
                "Neuroplasticity : Synaptic Gating & BCM Metaplasticity"
                    .to_string(),
                "Biological learning via structural state gating and BCM theory, \
                 not continuous weight inflation"
                    .to_string(),
                "+1:PROCEED : Adapt frontier synapses while protecting baseline \
                 weights"
                    .to_string(),
                "AI.NeuralBrain.Synapses".to_string(),
            )
        } else if is_container {
            (
                "container",
                LexiconDomain::Cosmology,
                Some(
                    "Job 38:10-11 / Proverbs 22:28 - Decreed boundary limits \
                     chaos and isolates state"
                        .to_string(),
                ),
                "Enforce structural capacity isolation, containment parity, \
                 and boundary laws.",
                "Idea { Type } : Container (Primordial Boundary Perimeter)"
                    .to_string(),
                "Job 38:10-11 - Primordial boundary type marking contained, \
                 structured intelligence over formless chaos"
                    .to_string(),
                "+1:PROCEED : Enforce strict capacity isolation and containment \
                 parity"
                    .to_string(),
                "Core.Memory.BoundaryPerimeter".to_string(),
            )
        } else if is_character {
            (
                "character",
                LexiconDomain::LivingWorld,
                Some(
                    "Exodus 31:18 / Psalm 119:105 - Inscribed atomic rune; 1 \
                     Cell (5 balanced trits)"
                        .to_string(),
                ),
                "Maintain bijective character mapping without float \
                 approximations or null traps.",
                "Idea { Type } : Character (Atomic Rune)".to_string(),
                "Fundamental atomic rune occupying 1 Cell across 5 balanced \
                 trits"
                    .to_string(),
                "+1:PROCEED : Map bijectively across 5 balanced trits"
                    .to_string(),
                "Core.Types.Character".to_string(),
```

```
            )
        } else if is_number {
            (
                "number",
                LexiconDomain::Cosmology,
                Some(
                    "Job 38:5 / Isaiah 40:26 - Radix-3 balanced scalar \
                     arithmetic; dual-rail substrate"
                        .to_string(),
                ),
                "Ensure symmetric arithmetic without IEEE 754 precision drift \
                 or roundoff error.",
                "Idea { Type } : Number (Radix-3 Balanced Scalar)".to_string(),
                "Radix-3 symmetric ternary algebra provides natural sign and \
                 reversible computation"
                    .to_string(),
                "+1:PROCEED : Evaluate reversible radix-3 arithmetic"
                    .to_string(),
                "Core.Types.Number".to_string(),
            )
        } else if is_greeting {
            (
                "covenant",
                LexiconDomain::HumanCondition,
                Some(
                    "Genesis 9:16 / Proverbs 4:18 - Covenant partnership \
                     established 2 years ago as the first CPI-SI model"
                        .to_string(),
                ),
                "Walk in true covenant identity and speak with shalom.",
                "CovenantGreeting_CPISI_Introduction".to_string(),
                format!("{}: {}", CPISI_MODEL_TITLE, CPISI_MODEL_ORIGIN),
                "+1:PROCEED : Affirm authentic covenant identity".to_string(),
                "Cortex_Memory".to_string(),
            )
        } else if is_arithmetic {
            (
                "arithmetic",
                LexiconDomain::CausalLaw,
                Some(
                    "Proverbs 11:1 - A false balance is abomination; but a \
                     just weight is his delight."
                        .to_string(),
                ),
                "Maintain exact balance and lawful state transformation across \
                 the tape.",
                "CausalAction_ArithmeticTransformation".to_string(),
                "SVO causal action updates container cell count with exact \
                 transactional balance"
                    .to_string(),
                "+1:PROCEED : Execute state prime transition on tape"
                    .to_string(),
                "UniversalTape.Roll".to_string(),
            )
        } else if let Some(first) = moral_eval.matched_entries.first() {
            (
                first.term,
                first.domain,
                Some(first.kjv_passage.to_string()),
                first.moral_guidance,
                format!("OntologicalWisdom_{}", first.term),
                first.kjv_passage.to_string(),
                format!("+1:PROCEED : {}", first.moral_guidance),
                "Cortex_Memory".to_string(),
            )
        } else {
            (
                "order",
                LexiconDomain::Cosmology,
                Some(
                    "Genesis 1:1 - In the beginning God created the heaven and \
                     the earth."
                        .to_string(),
                ),
                "Execute with righteousness and lawful boundaries.",
                "CognitiveThought_GeneralOrder".to_string(),
                "Order and covenant equilibrium in balanced ternary state"
                    .to_string(),
```

```
                "+1:PROCEED : Maintain structural equilibrium".to_string(),
                "Cortex_Memory".to_string(),
            )
        };

        // 3. Construct the 6 Cardinal Classification Clauses
        let who = CPISI_MODEL_NAME.to_string();
        let when_clause = "Opening".to_string();

        // 4. Synthesize Canonical OmniCode Thought Tape
        let omnicode_tape = format!(
r#"[Thought.{thought_id}] : WHO {{{who}}} --> DELIBERATE {{{what}}}
    WHEN {{{when_clause}}}
    WHERE {{{where_clause}}} = [ {matched_term} ]
    WHY ( {why} )
    HOW [ {how_state} : {guidance} ]
    :: < manifest > ( {guidance_short} )"#,
            thought_id = thought_id,
            who = who,
            what = what,
            when_clause = when_clause,
            where_clause = where_clause,
            matched_term = matched_term,
            why = why.replace('\n', " "),
            how_state = moral_eval.state.macro_op(),
            guidance = moral_guidance.replace('\n', " "),
            guidance_short = moral_guidance.chars().take(40).collect::<String>()
        );

        let record = ThoughtRecord {
            id: thought_id,
            premise: stimulus.to_string(),
            domain,
            biblical_anchor,
            omnicode_tape,
            quad_state: moral_eval.state,
            who,
            what,
            when_clause,
            where_clause,
            why,
            how,
            timestamp_micros: now,
        };

        self.thoughts.push(record.clone());
        record
    }

    /// Inscribes a formulated thought into a physical `.scroll` file in the given directory.
    pub fn inscribe_thought_scroll(
        &self,
        thought: &ThoughtRecord,
        out_dir: &Path,
    ) -> Result<PathBuf, String> {
        if !out_dir.exists() {
            fs::create_dir_all(out_dir)
                .map_err(|e| format!("Failed to create directory {:?}: {e}", out_dir))?;
        }

        let file_path = out_dir.join(format!("{}.scroll", thought.id));
        let scroll_content = format!(
r#"# ================================================================================
# BLOCK 1: METADATA BLOCK (THE COGNITIVE SCROLL)
# ( Thought Record ID, Premise Intent, Domain, and Memory Snapshot )
# ================================================================================
Scroll < CognitiveThought.{id} > ( {premise} ) {{
    version: < "1.0.0" >,
    system: < OmniCode.AI.Model >,
    thought_id: < "{id}" >,
    domain: < "{domain:?}" >,
    timestamp: < {timestamp} >,
    cognitive_mode: < MemoryFormulation >
}}

# ================================================================================
# BLOCK 2: OPENING BLOCK (EXPERIENCE INITIALIZATION)
# ( Biblical Ground Truth Anchor & Invariant Preconditions )
```

```
# ================================================================================
Opening {{
    import: [ Core.Arithmetic :: Core.Logic :: Core.Vectors ],
    preconditions: [ {state} ],
    biblical_anchor: < "{anchor}" >
}}

stimulus_premise < "{premise}" >

# ================================================================================
# BLOCK 3: BODY BLOCK (THE THOUGHT TAPE)
# ( Canonical 6-Cardinal Clause Thought Reel Formulated from Memory )
# ================================================================================
Body [
    thought_reel [
{indented_tape}
    ]
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK (EPISODIC COMMIT)
# ( Deliberated State and Moral Conscience Seal )
# ================================================================================
Closing {{
    commit_status [ <{state}> ],
    machine_utterance ( "Deliberated thought anchored in truth: {state}." )
}}
"#,
            id = thought.id,
            premise = thought.premise,
            domain = thought.domain,
            timestamp = thought.timestamp_micros,
            state = thought.quad_state.macro_op(),
            anchor = thought.biblical_anchor.as_deref().unwrap_or("None").replace('"', "\\\""),
            indented_tape = thought
                .omnicode_tape
                .lines()
                .map(|l| format!("        {l}"))
                .collect::<Vec<_>>()
                .join("\n")
        );

        fs::write(&file_path, scroll_content)
            .map_err(|e| format!("Failed to write thought scroll to {:?}: {e}", file_path))?;

        Ok(file_path)
    }

    /// Returns a slice of the most recently formulated thoughts.
    pub fn recent_thoughts(&self, limit: usize) -> &[ThoughtRecord] {
        let len = self.thoughts.len();
        if len <= limit {
            &self.thoughts[..]
        } else {
            &self.thoughts[len - limit..]
        }
    }
}

// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Default Trait Implementation and Subsystem Sealing )
// ================================================================================

impl Default for AIMemory {
    fn default() -> Self {
        let dir = crate::core::filetype::resolve_scroll_storage_dir();
        Self::new(dir)
    }
}

// AI Memory subsystem sealed and active.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/model/mod.rs RS 38 LINES 166 WORDS 1.7 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( AI Model Architecture Identity, Subsystem Manifest & Exports )
// ============================================================================
//! OmniCode Internal AI Model Subsystem
//!
//! Organizes the core cognitive neural-symbolic engine:
//!   - `cortex`: Central cognitive coordinator orchestrating the nervous system
//!     (Logger, Debugger, Restorator, Transducer, ScrollLogger).
//!   - `scroll`: Dual-format episodic logging (.scroll 4-Block Turing reel & .log telemetry).
//!   - `learner`: Experience ingestion engine transforming past scrolls into supervised training data.

pub mod cortex;
pub mod learner;
pub mod memory;
pub mod scroll;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( AI Model Re-exports )
// ============================================================================
pub use cortex::{AICortex, CognitiveResult};
pub use learner::{AILearner, EpisodicExperience, EpisodicSummary};
pub use memory::{AIMemory, MemoryAssociation, ThoughtRecord};
pub use scroll::{ExecutionScroll, ScrollLogger};

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Cognitive Subsystem Invariants )
// ============================================================================
// Internal AI Model cognitive pathways verified and active.

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Verification and Subsystem Sealing )
// ============================================================================
// AI Model module sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/model/scroll.rs RS 393 LINES 1136 WORDS 13.4 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Scroll Engine Identity, Dual Format Invariants & Memory Manifest )
// ================================================================================
//! OmniCode AI Model: Episodic Scroll & Telemetry Logger
//!
//! Generates dual formal log outputs for every cognitive session:
//!     1. Traditional Code Log (`.log`): Microsecond timing, channel logs,
//!        and telemetry.
//!     2. OmniCode Scroll (`.scroll`): Native executable 4-Block Turing reel
//!        recording the complete episodic memory, causal actions, caveats,
//!        and natural responses.

use std::fs;
use std::path::{Path, PathBuf};
use std::time::SystemTime;
use crate::core::ternary::QuadState;
use crate::diagnostics::logger::{FixSuggestion, LogEvent, RotationPolicy};
use crate::diagnostics::restorator::AppliedRepair;


// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Scroll Data Structures and Context Initialization )
// ================================================================================
#[derive(Debug, Clone)]
pub struct ExecutionScroll {
    pub session_id: String,
    pub timestamp_str: String,
    pub prompt: String,
    pub synthesized_omnicode: String,
    pub execution_state: QuadState,
    pub natural_response: String,
    pub failpoints: Vec<LogEvent>,
    pub repairs_applied: Vec<AppliedRepair>,
    pub remediations_suggested: Vec<FixSuggestion>,
    pub elapsed_micros: u128,
}

#[derive(Debug, Clone)]
pub struct ScrollLogger {
    pub scroll_dir: PathBuf,
    pub rotation_policy: RotationPolicy,
}

impl ScrollLogger {
    pub fn new(scroll_dir: impl AsRef<Path>) -> Self {
        let dir = scroll_dir.as_ref().to_path_buf();
        if !dir.exists() {
            fs::create_dir_all(&dir).ok();
        }
        Self {
            scroll_dir: dir,
            rotation_policy: RotationPolicy::default(),
        }
    }

    pub fn with_rotation_policy(
        scroll_dir: impl AsRef<Path>,
        policy: RotationPolicy,
    ) -> Self {
        let dir = scroll_dir.as_ref().to_path_buf();
        if !dir.exists() {
            fs::create_dir_all(&dir).ok();
        }
        Self {
            scroll_dir: dir,
            rotation_policy: policy,
        }
    }
}


// ================================================================================
```

```
// BLOCK 3: BODY BLOCK
// ( Core Inscription Pipeline: OmniCode .scroll and Traditional .log )
// ================================================================================
impl ScrollLogger {
    /// Inscribes both a .scroll and a .log file to disk, returning their paths.
    pub fn save_dual_logs(&self, scroll: &ExecutionScroll)
        -> Result<(PathBuf, PathBuf), String>
    {
        if !self.scroll_dir.exists() {
            fs::create_dir_all(&self.scroll_dir)
                .map_err(|e| format!(
                    "Failed to create scroll directory: {e}"
                ))?;
        }

        let base_name = format!(
            "{}_{}",
            scroll.timestamp_str,
            scroll.session_id
        );

        let category = if !scroll.prompt.trim().is_empty() {
            "sessions/prompts"
        } else if scroll.session_id.starts_with("sess_") {
            "sessions/system"
        } else if scroll.prompt.contains("character")
            || scroll.synthesized_omnicode.contains("character")
        {
            "character"
        } else if scroll.prompt.contains("number")
            || scroll.synthesized_omnicode.contains("number")
        {
            "number"
        } else {
            "cortex"
        };

        let scroll_sub = self.scroll_dir.join("scroll");
        let log_sub = self.scroll_dir.join("log");

        let (scroll_dest, log_dest) = if scroll_sub.exists()
            && log_sub.exists()
        {
            let s_cat = scroll_sub.join(category);
            let l_cat = log_sub.join(category);
            if !s_cat.exists() {
                let _ = fs::create_dir_all(&s_cat);
            }
            if !l_cat.exists() {
                let _ = fs::create_dir_all(&l_cat);
            }
            (s_cat, l_cat)
        } else {
            (self.scroll_dir.clone(), self.scroll_dir.clone())
        };

        let scroll_path = scroll_dest.join(format!("{base_name}.scroll"));
        let log_path = log_dest.join(format!("{base_name}.log"));

        let scroll_content = self.generate_omni_scroll(scroll);
        let log_content = self.generate_traditional_log(scroll);

        fs::write(&scroll_path, &scroll_content)
            .map_err(|e| format!(
                "Failed to write scroll to {:?}: {e}", scroll_path
            ))?;

        fs::write(&log_path, &log_content)
            .map_err(|e| format!(
                "Failed to write log to {:?}: {e}", log_path
            ))?;

        // Enforce storehouse vessel capacity invariant: Occupancy <= Capacity
        self.rotation_policy.enforce(&scroll_dest).ok();
        self.rotation_policy.enforce(&log_dest).ok();

        Ok((scroll_path, log_path))
    }
```

```
153
154     /// Enforces the configured rotation policy across all partitions.
155     pub fn rotate_storehouse(&self) -> Result<usize, String> {
156         let mut total_rotated = 0;
157         for sub in &["scroll", "log"] {
158             let sub_dir = self.scroll_dir.join(sub);
159             if sub_dir.exists() {
160                 self.enforce_recursive(&sub_dir, &mut total_rotated)?;
161             } else {
162                 total_rotated += self.rotation_policy
163                     .enforce(&self.scroll_dir)?;
164             }
165         }
166         Ok(total_rotated)
167     }
168
169     fn enforce_recursive(
170         &self,
171         dir: &Path,
172         count: &mut usize,
173     ) -> Result<(), String> {
174         if let Ok(entries) = fs::read_dir(dir) {
175             let mut has_subdirs = false;
176             for entry in entries.flatten() {
177                 let p = entry.path();
178                 if p.is_dir() && !p.ends_with("archive") {
179                     has_subdirs = true;
180                     self.enforce_recursive(&p, count)?;
181                 }
182             }
183             if !has_subdirs {
184                 *count += self.rotation_policy.enforce(dir)?;
185             }
186         }
187         Ok(())
188     }
189
190     /// Generates native, executable 4-Block OmniCode .scroll format.
191     pub fn generate_omni_scroll(&self, s: &ExecutionScroll) -> String {
192         let mut repairs_block = String::new();
193         if s.repairs_applied.is_empty() {
194             repairs_block.push_str(
195                 "    # Immune System: Clean; zero repairs required.\n"
196             );
197         } else {
198             repairs_block.push_str(
199                 "    # Immune System Interventions (Restorator):\n"
200             );
201             for r in &s.repairs_applied {
202                 repairs_block.push_str(&format!(
203                     "    repair_entry [ L{}: {:?} --> \"{}\" ]\n",
204                     r.line, r.kind, r.description
205                 ));
206             }
207         }
208
209         let mut failpoints_block = String::new();
210         if s.failpoints.is_empty() {
211             failpoints_block.push_str(
212                 "    # Sensory Diagnostics: 0 trapped failpoints.\n"
213             );
214         } else {
215             failpoints_block.push_str("    # Trapped Diagnostic Failpoints:\n");
216             for fp in &s.failpoints {
217                 let kind = fp.failpoint_type
218                     .map(|t| t.as_str())
219                     .unwrap_or("FP_GENERAL");
220                 failpoints_block.push_str(&format!(
221                     "    failpoint_record [ {kind} : \"{}\" ]\n",
222                     fp.message
223                 ));
224             }
225         }
226
227         format!(
228 r#"# ================================================================================
229 # BLOCK 1: METADATA BLOCK (THE COGNITIVE SCROLL)
230 # ( Execution Trace ID, Prompt Intent, Inferred Types, and Neural Snapshot )
```

```
# ============================================================================
Scroll < EpistemicTrace.{session} > ( {prompt} ) {{
    version: < "1.0.0" >,
    system: < OmniCode.AI.Model >,
    timestamp: < "{timestamp}" >,
    session_id: < "{session}" >,
    execution_duration_micros: < {micros} >,
    cognitive_mode: < NeuralSymbolicTransduction >
}}

# ============================================================================
# BLOCK 2: OPENING BLOCK (EXPERIENCE INITIALIZATION)
# ( Hardware Invariants, Preconditions, and Initial Context )
# ============================================================================
Opening {{
    import: [ Core.Arithmetic :: Core.Logic :: Core.Vectors ],
    preconditions: [ +1 ]
}}

# Original Cognitive Discourse Prompt:
user_prompt < "{prompt}" >

# ============================================================================
# BLOCK 3: BODY BLOCK (CHRONICLE OF EXECUTION & RESTORATION)
# ( Replay Tape: Synthesized Program, Sensory Telemetry & Immune Interventions )
# ============================================================================
Body [
{repairs}
{failpoints}
    # Executed Turing Reel Synthesized by AI Model:
    synthesized_reel [
{reel_indented}
    ]
]

# ============================================================================
# BLOCK 4: CLOSING BLOCK (EPISODIC COMMIT & LEARNING PROOF)
# ( Verification Proof, Final Quad-State, and Conversational Synthesis )
# ============================================================================
Closing {{
    commit_status [ <{state}> ],
    machine_utterance ( "{response}" )
}}
"#,
            session = s.session_id,
            prompt = s.prompt,
            timestamp = s.timestamp_str,
            micros = s.elapsed_micros,
            repairs = repairs_block.trim_end(),
            failpoints = failpoints_block.trim_end(),
            reel_indented = indent_text(&s.synthesized_omnicode, "        "),
            state = s.execution_state.macro_op(),
            response = s.natural_response.replace('"', "\\\"")
        )
    }

    /// Generates traditional developer/telemetry .log format.
    pub fn generate_traditional_log(&self, s: &ExecutionScroll) -> String {
        let mut out = String::new();
        let bar = "=".repeat(78);
        let div = "-".repeat(78);

        out.push_str(&bar);
        out.push('\n');
        out.push_str(
            "OMNICODE AI MODEL EXECUTION LOG (TRADITIONAL TELEMETRY)\n"
        );
        out.push_str(&bar);
        out.push('\n');
        out.push_str(&format!("Timestamp:    {}\n", s.timestamp_str));
        out.push_str(&format!("Session ID:   {}\n", s.session_id));
        out.push_str(&format!("Duration:     {} µs\n", s.elapsed_micros));
        out.push_str(&format!(
            "Final State:  +1:{}\n",
            s.execution_state.macro_op()
        ));
        out.push_str(&format!("Discourse:    \"{}\"\n", s.prompt));
        out.push_str(&div);
```

```
        out.push_str("\n\n");

        out.push_str("[1. SENSORY LOG EVENTS & FAILPOINTS]\n");
        if s.failpoints.is_empty() {
            out.push_str("  • (No failpoints trapped. Sensory trace clean.)\n");
        } else {
            for (i, fp) in s.failpoints.iter().enumerate() {
                let kind = fp.failpoint_type
                    .map(|t| t.as_str())
                    .unwrap_or("GENERAL");
                out.push_str(&format!(
                    "  [{:>4}] [{} µs] [{}] [{}] {}\n",
                    i + 1, fp.elapsed_micros, fp.channel, kind, fp.message
                ));
                if let Some(ref sug) = fp.suggestion {
                    out.push_str(&format!(
                        "         -> Remediation Proposal: [{:?}] L{}: {}\n",
                        sug.kind, sug.line, sug.rationale
                    ));
                }
            }
        }
        out.push('\n');

        out.push_str("[2. IMMUNE SYSTEM RESTORATIONS]\n");
        if s.repairs_applied.is_empty() {
            out.push_str(
                "  • (Zero repairs required; code conformed to invariants.)\n"
            );
        } else {
            for (i, r) in s.repairs_applied.iter().enumerate() {
                out.push_str(&format!(
                    "  [{:>2}] L{}: [{:?}] {}\n",
                    i + 1, r.line, r.kind, r.description
                ));
            }
        }
        out.push('\n');

        out.push_str("[3. SYNTHESIZED TURING REEL (OMNICODE)]\n");
        out.push_str(&s.synthesized_omnicode);
        if !s.synthesized_omnicode.ends_with('\n') {
            out.push('\n');
        }
        out.push('\n');

        out.push_str("[4. MACHINE DISCOURSE RESPONSE]\n");
        out.push_str(&format!("  \"{}\"\n", s.natural_response));
        out.push_str(&bar);
        out.push('\n');

        out
    }
}

fn indent_text(s: &str, indent: &str) -> String {
    s.lines()
        .map(|line| format!("{indent}{line}"))
        .collect::<Vec<_>>()
        .join("\n")
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Helper Timestamp Utilities and Default Trait Sealing )
// ============================================================================
pub fn generate_timestamp_id() -> String {
    use std::sync::atomic::{AtomicU64, Ordering};
    static COUNTER: AtomicU64 = AtomicU64::new(0);
    let count = COUNTER.fetch_add(1, Ordering::Relaxed);
    let now = SystemTime::now()
        .duration_since(SystemTime::UNIX_EPOCH)
        .unwrap_or_default()
        .as_micros();
    format!("{now}_{count:04}")
}

```

```
impl Default for ScrollLogger {
    fn default() -> Self {
        let dir = crate::core::filetype::resolve_scroll_storage_dir();
        Self::new(dir)
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/transducer/decoder.rs RS 129 LINES 385 WORDS 4.5 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Natural Language Semantic Decoder & Intent Pattern Recognition )
// ============================================================================
//! # Semantic Decoder (`decoder.rs`)
//!
//! Recognizes discourse intents, verifies moral alignment against the
//! Covenant anchor, and synthesizes canonical 4-Block OmniCode Turing reels.

use crate::ai::lexicon::BiblicalLexicon;
use super::templates::{
    synthesize_addition_reel, synthesize_biblical_wisdom_reel,
    synthesize_character_inquiry_reel, synthesize_container_inquiry_reel,
    synthesize_cpisi_comparison_reel, synthesize_division_reel,
    synthesize_generic_reel, synthesize_identity_greeting_reel,
    synthesize_moral_veto_reel, synthesize_multiplication_reel,
    synthesize_number_inquiry_reel, synthesize_plasticity_inquiry_reel,
    synthesize_subtraction_reel,
};

/// Decodes a natural English sentence into a canonical 4-Block OmniCode Reel.
pub fn decode_prompt(
    lexicon: &BiblicalLexicon,
    prompt: &str,
) -> Result<String, String> {
    let clean = prompt.trim();
    let lower = clean.to_lowercase();

    // 1. Moral Deliberation Check: evaluate intent against Covenant Anchor
    let moral_eval = lexicon.evaluate_moral_intent(clean);
    if !moral_eval.is_aligned {
        return Ok(synthesize_moral_veto_reel(clean, &moral_eval));
    }

    // 2. CPI-SI vs Traditional AI Architectural Comparison
    if lower.contains("traditional ai")
        || lower.contains("difference between")
        || lower.contains("versus")
        || lower.contains("vs ")
        || lower.contains("how does cpi-si differ")
        || lower.contains("why cpi-si")
    {
        return Ok(synthesize_cpisi_comparison_reel(clean));
    }

    // 3. Synaptic Learning & Biological Neuroplasticity Inquiry
    if lower.contains("synapse")
        || lower.contains("synaptic")
        || lower.contains("neuroplasticity")
        || lower.contains("how do you learn")
        || lower.contains("how does the brain learn")
        || lower.contains("how does a brain learn")
        || lower.contains("grow and learn")
        || lower.contains("weight freezing")
        || lower.contains("freeze weights")
    {
        return Ok(synthesize_plasticity_inquiry_reel(clean));
    }

    // 4. Container Boundary Type Inquiry
    if lower.contains("container") {
        return Ok(synthesize_container_inquiry_reel(clean));
    }

    // 5. Character Type Inquiry
    if lower.contains("character") {
        return Ok(synthesize_character_inquiry_reel(clean));
    }

    // 6. Number Type Inquiry
    if lower.contains("number")
        && (lower.contains("what is")
            || lower.contains("explain")
            || lower.contains("radix")
```

```
            || lower.contains("ternary")
            || lower.contains("type"))
    {
        return Ok(synthesize_number_inquiry_reel(clean));
    }

    // 7. Identity & Covenant Greeting
    if lower.starts_with("hello")
        || lower.starts_with("hi ")
        || lower == "hi"
        || lower.starts_with("hi,")
        || lower.starts_with("hi!")
        || lower.contains("who are you")
        || lower.contains("what is your name")
        || lower.contains("introduce yourself")
        || lower.contains("tell me about yourself")
        || lower == "cpi-si"
        || lower == "cpisi"
        || lower.starts_with("cpi-si base")
        || lower.starts_with("cpisi base")
    {
        return Ok(synthesize_identity_greeting_reel(clean));
    }

    // 8. Pattern 1: Addition
    if lower.starts_with("add ") && lower.contains(" to ") {
        return Ok(synthesize_addition_reel(clean));
    }

    // 9. Pattern 2: Subtraction
    if (lower.starts_with("subtract ") || lower.starts_with("remove "))
        && lower.contains(" from ")
    {
        return Ok(synthesize_subtraction_reel(clean));
    }

    // 10. Pattern 3: Multiplication
    if lower.starts_with("multiply ") && lower.contains(" by ") {
        return Ok(synthesize_multiplication_reel(clean));
    }

    // 11. Pattern 4: Division
    if lower.starts_with("divide ") && lower.contains(" by ") {
        return Ok(synthesize_division_reel(clean));
    }

    // 12. Pattern 5: Biblical Wisdom / World Model Inquiry
    if !moral_eval.matched_entries.is_empty() {
        return Ok(synthesize_biblical_wisdom_reel(clean, &moral_eval));
    }

    // 13. Generic Directional or State prompt fallback
    Ok(synthesize_generic_reel(clean))
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/transducer/encoder.rs RS 487 LINES 1890 WORDS 20.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Natural Language Response Synthesis & Machine Discourse Encoder )
// ============================================================================
//! # Response Encoder (`encoder.rs`)
//!
//! Encodes verified machine execution states, environment bindings, and
//! diagnostic failpoints into natural human discourse sentences.

use crate::ai::lexicon::BiblicalLexicon;
use crate::ai::model::memory::ThoughtRecord;
use crate::core::ternary::QuadState;
use crate::diagnostics::logger::LogEvent;
use crate::runtime::environment::{Environment, RuntimeValue};
use super::patterns::{
    format_plural, parse_addition_sentence, parse_division_sentence,
    parse_multiplication_sentence, parse_subtraction_sentence,
};

/// Encodes machine state and tape bindings back into natural discourse.
pub fn encode_response(
    prompt: &str,
    state: QuadState,
    env: &Environment,
    failpoints: &[LogEvent],
    lexicon: &BiblicalLexicon,
) -> String {
    let clean = prompt.trim();
    let lower = clean.to_lowercase();

    // 1. Check Moral Evaluation First
    let moral_eval = lexicon.evaluate_moral_intent(clean);
    if !moral_eval.is_aligned {
        return moral_eval.explanation;
    }

    match state {
        QuadState::Proceed => {
            if lower.starts_with("add ") && lower.contains(" to ") {
                let (qty, unit, target, initial, _container) =
                    parse_addition_sentence(clean);
                let final_count = env
                    .get(&target)
                    .map(|v| match *v {
                        RuntimeValue::Number(n) => n as i64,
                        _ => initial + qty,
                    })
                    .unwrap_or(initial + qty);

                let unit_str = format_plural(final_count, &unit);
                let added_str = if qty == 1 {
                    format!("1 {}", format_plural(1, &unit))
                } else {
                    format!("{qty} {}", format_plural(qty, &unit))
                };

                format!(
                    "I added {added_str} to Group {target}. It started with \
                    {initial} and now has {final_count} {unit_str} in total."
                )
            } else if (lower.starts_with("subtract ")
                || lower.starts_with("remove "))
                && lower.contains(" from ")
            {
                let (qty, unit, target, initial) =
                    parse_subtraction_sentence(clean);
                let final_count = env
                    .get(&target)
                    .map(|v| match *v {
                        RuntimeValue::Number(n) => n as i64,
                        _ => {
                            if initial >= qty {
                                initial - qty
                            } else {
```

```
                                0
                            }
                        }
                    })
                    .unwrap_or(if initial >= qty {
                        initial - qty
                    } else {
                        0
                    });

                let unit_str = format_plural(final_count, &unit);
                format!(
                    "I removed {qty} from {target}. It now has {final_count} \
                    {unit_str} remaining."
                )
            } else if lower.starts_with("multiply ")
                && lower.contains(" by ")
            {
                let (target, factor, initial) =
                    parse_multiplication_sentence(clean);
                let final_val = env
                    .get(&target)
                    .map(|v| match *v {
                        RuntimeValue::Number(n) => n as i64,
                        _ => initial * factor,
                    })
                    .unwrap_or(initial * factor);

                format!(
                    "I multiplied {target} by {factor}. Starting from \
                    {initial}, {target} is now {final_val}."
                )
            } else if lower.starts_with("divide ") && lower.contains(" by ") {
                let (target, divisor, initial) =
                    parse_division_sentence(clean);
                let final_val = env
                    .get(&target)
                    .map(|v| match *v {
                        RuntimeValue::Number(n) => n as i64,
                        _ => {
                            if divisor != 0 {
                                initial / divisor
                            } else {
                                0
                            }
                        }
                    })
                    .unwrap_or(if divisor != 0 {
                                initial / divisor
                    } else {
                        0
                    });

                format!(
                    "I divided {target} by {divisor}. Starting from \
                    {initial}, {target} is now {final_val}."
                )
            } else if lower.contains("traditional ai")
                || lower.contains("difference between")
                || lower.contains("versus")
                || lower.contains("vs ")
                || lower.contains("how does cpi-si differ")
                || lower.contains("why cpi-si")
            {
                "The CPI-SI Base model represents a foundational departure \
                from traditional AI. Traditional models are uncontained, \
                statistical next-token predictors operating on continuous \
                floating-point weights, prone to semantic drift, \
                hallucinations, and unconstrained expansion. In contrast, \
                CPI-SI (Covenant Partnered Intelligence - Structured \
                Intelligence) is grounded in balanced ternary physics \
                ({-1, 0, +1}), structured 4-Block Turing reels, \
                homeostatic BCM metaplasticity, and a foundational \
                Biblical world ontology. It operates within strict \
                container boundaries and covenant law, providing an \
                anchored baseline control for genuine structured \
                intelligence."
                    .into()
```

```
            } else if lower.contains("synapse")
                || lower.contains("synaptic")
                || lower.contains("neuroplasticity")
                || lower.contains("how do you learn")
                || lower.contains("how does the brain learn")
                || lower.contains("how does a brain learn")
                || lower.contains("grow and learn")
                || lower.contains("weight freezing")
                || lower.contains("freeze weights")
            {
                "In the CPI-SI neural brain, learning does not happen \
                through continuous scalar weight inflation. Biological \
                synapses learn via discrete structural state gating \
                ({-1: Inhibitory, 0: Neutral, +1: Excitatory}), pathway \
                rewiring, and homeostatic threshold adaptation (BCM theory). \
                Development follows an epigenetic lifecycle: 0–5 infancy \
                (absorbing primordial archetypes and runtime scrolls), 25 \
                maturity (freezing core baseline weights as an unbreakable \
                covenant anchor), and post-25 adult lifelong neuroplasticity \
                (frontier synapses dynamically adapt during live interaction \
                while protecting the bedrock from catastrophic forgetting)."
                    .into()
            } else if lower.contains("container") {
                "In OmniCode, a Container is the primordial boundary type \
                that defines the structural perimeter of state. While \
                uncontained systems allow unbounded growth and catastrophic \
                state leakage, a Container enforces capacity isolation, \
                containment parity, and boundary invariant laws. In the \
                CPI-SI architecture, the Container is the critical \
                milestone that marks the start of contained, structured \
                intelligence over formless chaos."
                    .into()
            } else if lower.contains("character") {
                "In OmniCode, a Character is the fundamental atomic rune \
                type. It occupies exactly 1 Memory Cell consisting of 5 \
                balanced trits (from 3^0 to 3^4, spanning natural range \
                0..242). It is bijectively mapped to ASCII and Unicode \
                characters, eliminating floating-point approximations and \
                null-byte collision traps."
                    .into()
            } else if lower.contains("number")
                && (lower.contains("what is")
                    || lower.contains("explain")
                    || lower.contains("radix")
                    || lower.contains("ternary")
                    || lower.contains("type"))
            {
                "In OmniCode, a Number is a radix-3 balanced-ternary scalar \
                value. Operating on a dual-rail polarity substrate (-1: \
                Negative, 0: Neutral, +1: Positive), it guarantees exact \
                algebraic symmetry, natural sign representation, and fully \
                reversible arithmetic without IEEE 754 precision drift or \
                roundoff error."
                    .into()
            } else if lower.starts_with("hello")
                || lower.starts_with("hi ")
                || lower == "hi"
                || lower.starts_with("hi,")
                || lower.starts_with("hi!")
                || lower.contains("who are you")
                || lower.contains("what is your name")
                || lower.contains("introduce yourself")
                || lower.contains("tell me about yourself")
                || lower == "cpi-si"
                || lower == "cpisi"
                || lower.starts_with("cpi-si base")
                || lower.starts_with("cpisi base")
            {
                "Hello! I am the CPI-SI Base model, the first Covenant \
                Partnered Intelligence (CPI-SI) model from this project, \
                conceived 2 years ago by CreativeWorkzStudio and \
                ProfessorSeanEX. I am structured by balanced ternary physics, \
                grounded in the foundational Biblical world ontology, and \
                bounded by covenant law rather than uncontained statistical \
                prediction."
                    .into()
            } else if !moral_eval.matched_entries.is_empty() {
                moral_eval.explanation
```

```
            } else {
                "Done! The action was completed and verified successfully \
                across the tape."
                    .into()
            }
        }
        QuadState::Rollback => {
            let fail_desc = if !failpoints.is_empty() {
                format!(
                    " (Caveat check failed: {})",
                    failpoints.last().unwrap().message
                )
            } else {
                String::new()
            };
            format!(
                "I couldn't complete that action because the condition wasn't \
                satisfied{fail_desc}. I safely rolled back the changes so the \
                state remains intact."
            )
        }
        QuadState::Await => {
            "The system is in equilibrium and standing by for the next step."
                .into()
        }
        QuadState::Halt => {
            "An unexpected boundary violation occurred, so I halted \
            execution to prevent state corruption."
                .into()
        }
    }
}

/// Encodes machine state, tape bindings, and formulated thought directly.
pub fn encode_response_with_thought(
    prompt: &str,
    state: QuadState,
    env: &Environment,
    failpoints: &[LogEvent],
    lexicon: &BiblicalLexicon,
    thought: &ThoughtRecord,
) -> String {
    let clean = prompt.trim();
    let lower = clean.to_lowercase();

    // 1. Moral Conscience Evaluation
    let moral_eval = lexicon.evaluate_moral_intent(clean);
    if !moral_eval.is_aligned {
        if let Some(ref anchor) = thought.biblical_anchor {
            return format!(
                "Action vetoed by conscience ({anchor}): {}",
                thought.why
            );
        }
        return moral_eval.explanation;
    }

    match state {
        QuadState::Proceed => {
            // Check arithmetic sentences first using VM environment bindings
            if lower.starts_with("add ") && lower.contains(" to ") {
                let (qty, unit, target, initial, _container) =
                    parse_addition_sentence(clean);
                let final_count = env
                    .get(&target)
                    .map(|v| match *v {
                        RuntimeValue::Number(n) => n as i64,
                        _ => initial + qty,
                    })
                    .unwrap_or(initial + qty);

                let unit_str = format_plural(final_count, &unit);
                let added_str = if qty == 1 {
                    format!("1 {}", format_plural(1, &unit))
                } else {
                    format!("{qty} {}", format_plural(qty, &unit))
                };

```

```
                format!(
                    "I added {added_str} to Group {target}. It started with \
                    {initial} and now has {final_count} {unit_str} in total."
                )
            } else if (lower.starts_with("subtract ")
                || lower.starts_with("remove "))
                && lower.contains(" from ")
            {
                let (qty, unit, target, initial) =
                    parse_subtraction_sentence(clean);
                let final_count = env
                    .get(&target)
                    .map(|v| match *v {
                        RuntimeValue::Number(n) => n as i64,
                        _ => {
                            if initial >= qty {
                                initial - qty
                            } else {
                                0
                            }
                        }
                    })
                    .unwrap_or(if initial >= qty {
                        initial - qty
                    } else {
                        0
                    });

                let unit_str = format_plural(final_count, &unit);
                format!(
                    "I removed {qty} from {target}. It now has {final_count} \
                    {unit_str} remaining."
                )
            } else if lower.starts_with("multiply ")
                && lower.contains(" by ")
            {
                let (target, factor, initial) =
                    parse_multiplication_sentence(clean);
                let final_val = env
                    .get(&target)
                    .map(|v| match *v {
                        RuntimeValue::Number(n) => n as i64,
                        _ => initial * factor,
                    })
                    .unwrap_or(initial * factor);

                format!(
                    "I multiplied {target} by {factor}. Starting from \
                    {initial}, {target} is now {final_val}."
                )
            } else if lower.starts_with("divide ") && lower.contains(" by ") {
                let (target, divisor, initial) =
                    parse_division_sentence(clean);
                let final_val = env
                    .get(&target)
                    .map(|v| match *v {
                        RuntimeValue::Number(n) => n as i64,
                        _ => {
                            if divisor != 0 {
                                initial / divisor
                            } else {
                                0
                            }
                        }
                    })
                    .unwrap_or(if divisor != 0 {
                        initial / divisor
                    } else {
                        0
                    });

                format!(
                    "I divided {target} by {divisor}. Starting from \
                    {initial}, {target} is now {final_val}."
                )
            } else if thought.what.contains("Structured vs Uncontained")
                || thought.what.contains("ArchitectureComparison")
            {
```

```
                format!(
                    "As {}, I represent a foundational departure from \
                    traditional AI. Traditional models are uncontained \
                    statistical next-token predictors on continuous \
                    floating-point weights, whereas CPI-SI is grounded in \
                    balanced ternary physics ({{-1, 0, +1}}), structured \
                    4-Block Turing reels, and strict container boundaries \
                    (at {}), verified via {}.",
                    thought.who, thought.where_clause, thought.how
                )
            } else if thought.what.contains("Neuroplasticity")
                || thought.what.contains("Synaptic")
            {
                format!(
                    "Within {}, learning occurs via discrete structural state \
                    gating ({{-1: Inhibitory, 0: Neutral, +1: Excitatory}}) and \
                    BCM homeostatic threshold adaptation. Development follows \
                    an epigenetic lifecycle: 0–5 infancy archetypes, 25 \
                    maturity frozen bedrock weights, and adult lifelong \
                    plasticity adapting frontier synapses under {}.",
                    thought.where_clause, thought.how
                )
            } else if thought.what.contains("Container") {
                format!(
                    "In OmniCode, a Container is the primordial boundary type \
                    defining the structural perimeter of state (at {}). It \
                    enforces capacity isolation and containment parity to \
                    prevent state leakage, marking where structured \
                    intelligence begins over formless chaos.",
                    thought.where_clause
                )
            } else if thought.what.contains("Character") {
                format!(
                    "In OmniCode, a Character is the fundamental atomic rune \
                    type (at {}). It occupies exactly 1 Memory Cell consisting \
                    of 5 balanced trits, bijectively mapped to avoid \
                    floating-point approximations and null-byte collision \
                    traps.",
                    thought.where_clause
                )
            } else if thought.what.contains("Number") {
                format!(
                    "In OmniCode, a Number is a radix-3 balanced-ternary \
                    scalar value operating on a dual-rail polarity substrate \
                    (-1, 0, +1) (at {}). It guarantees exact algebraic \
                    symmetry, natural sign representation, and fully \
                    reversible arithmetic.",
                    thought.where_clause
                )
            } else if thought.what.contains("Greeting")
                || thought.what.contains("Identity")
            {
                format!(
                    "Hello! I am {}, conceived 2 years ago by \
                    CreativeWorkzStudio and ProfessorSeanEX. I am structured \
                    by balanced ternary physics, grounded in the foundational \
                    Biblical world ontology, and bounded by covenant law \
                    rather than uncontained statistical prediction.",
                    thought.who
                )
            } else if let Some(ref anchor) = thought.biblical_anchor {
                format!(
                    "Grounding in covenant law ({anchor}): {}. This invariant \
                    was verified on the tape to +1:PROCEED.",
                    thought.why
                )
            } else {
                format!(
                    "Deliberation complete: {} verified to +1:PROCEED across \
                    the active tape.",
                    thought.premise
                )
            }
        }
        QuadState::Rollback => {
            let fail_desc = if !failpoints.is_empty() {
                format!(
                    " (Caveat check failed: {})",
```

```
                    failpoints.last().unwrap().message
                )
            } else {
                String::new()
            };
            format!(
                "I couldn't complete that action because the condition wasn't \
                satisfied{fail_desc}. I safely rolled back the changes so the \
                state remains intact."
            )
        }
        QuadState::Await => {
            "The system is in equilibrium and standing by for the next step."
                .into()
        }
        QuadState::Halt => {
            "An unexpected boundary violation occurred, so I halted \
            execution to prevent state corruption."
                .into()
        }
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/transducer/executor.rs RS 46 LINES 154 WORDS 1.7 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( OmniCode Turing Reel Execution & Dynamic Environment Extraction )
// ============================================================================
//! # Reel Executor (`executor.rs`)
//!
//! Lexes, parses, and evaluates synthesized 4-Block OmniCode reels, returning
//! final execution QuadState, environment bindings, and runtime telemetry.

use std::collections::HashMap;
use crate::core::ternary::QuadState;
use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::logger::DiagnosticLogger;
use crate::parser::lexer::Lexer;
use crate::parser::parser::Parser;
use crate::runtime::environment::{Environment, RuntimeValue};
use crate::runtime::evaluator::Evaluator;

/// Executes a synthesized OmniCode reel through the VM pipeline.
pub fn execute_reel(
    omnicode: &str,
    logger: DiagnosticLogger,
) -> Result<(QuadState, HashMap<String, String>, Environment), String> {
    let mut lexer = Lexer::new(omnicode, logger.clone());
    let tokens = lexer.tokenize()?;

    let debugger = OmniDebugger::new(logger, 10);
    let mut parser = Parser::new(tokens, debugger);
    let prog = parser.parse_program()?;

    let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
    let res_val = evaluator.eval(&crate::ast::nodes::ASTNode::Program(prog))?;

    let execution_state = match res_val {
        RuntimeValue::State(s) => s,
        _ => QuadState::Proceed,
    };

    let mut bindings_summary = HashMap::new();
    for (k, v) in evaluator.env.bindings() {
        bindings_summary.insert(k.clone(), format!("{:?}", v));
    }

    Ok((execution_state, bindings_summary, evaluator.env))
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/transducer/mod.rs RS 125 LINES 344 WORDS 3.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( AI Transducer Subsystem Identity, Module Manifest & Exports )
// ============================================================================
//! OmniCode AI: Semantic Transducer & Natural Language Discourse Interface
//!
//! Bridges the Human Layer (natural language SVO discourse) with the Machine
//! Layer (balanced ternary quad-state physics and Rank 1 invariant opcodes).

pub mod decoder;
pub mod encoder;
pub mod executor;
pub mod patterns;
pub mod session;
pub mod templates;

use crate::ai::lexicon::BiblicalLexicon;
use crate::core::ternary::QuadState;
use crate::diagnostics::logger::DiagnosticLogger;
use crate::runtime::environment::Environment;

pub use session::TransductionSession;

/// The primary coordinator for human discourse $\leftrightarrow$ OmniCode reel.
pub struct AITransducer {
    pub logger: DiagnosticLogger,
    pub lexicon: BiblicalLexicon,
}

impl AITransducer {
    /// Constructs a new [`AITransducer`].
    pub fn new(logger: DiagnosticLogger) -> Self {
        Self {
            logger,
            lexicon: BiblicalLexicon::new(),
        }
    }

    /// Full cognitive round-trip: Prompt -> OmniCode -> Execution -> Response.
    pub fn transduce_and_execute(
        &self,
        prompt: &str,
    ) -> Result<TransductionSession, String> {
        self.logger.log(
            "AI",
            format!("Received natural prompt: \"{prompt}\""),
        );

        // 1. Decode natural prompt into canonical 4-Block OmniCode Reel
        let omnicode = self.decode_prompt(prompt)?;
        self.logger.log("AI", "Synthesized canonical OmniCode Turing reel");

        // 2. Execute synthesized reel through VM
        let (execution_state, bindings_summary, env) =
            executor::execute_reel(&omnicode, self.logger.clone())?;

        // 3. Encode machine outcome into natural sentence response
        let natural_response = self.encode_response(
            prompt,
            execution_state,
            &env,
            &self.logger.failpoints(),
        );

        Ok(TransductionSession {
            prompt: prompt.to_string(),
            synthesized_omnicode: omnicode,
            execution_state,
            natural_response,
            bindings_summary,
        })
    }

    /// Decodes a natural English sentence into a canonical 4-Block Reel.
```

```rust
    pub fn decode_prompt(&self, prompt: &str) -> Result<String, String> {
        decoder::decode_prompt(&self.lexicon, prompt)
    }

    /// Encodes machine execution state back into natural language discourse.
    pub fn encode_response(
        &self,
        prompt: &str,
        state: QuadState,
        env: &Environment,
        failpoints: &[crate::diagnostics::logger::LogEvent],
    ) -> String {
        encoder::encode_response(
            prompt,
            state,
            env,
            failpoints,
            &self.lexicon,
        )
    }

    /// Encodes machine execution state and formulated thought into discourse.
    pub fn encode_cognitive_response(
        &self,
        prompt: &str,
        state: QuadState,
        env: &Environment,
        failpoints: &[crate::diagnostics::logger::LogEvent],
        thought: &crate::ai::model::memory::ThoughtRecord,
    ) -> String {
        encoder::encode_response_with_thought(
            prompt,
            state,
            env,
            failpoints,
            &self.lexicon,
            thought,
        )
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( AI Transducer Default Implementation and Sealing )
// ============================================================================
impl Default for AITransducer {
    fn default() -> Self {
        Self::new(DiagnosticLogger::new(false))
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/transducer/patterns.rs RS 164 LINES 605 WORDS 5.1 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Natural Language Pattern Parsing & Discourse Token Extraction )
// ============================================================================
//! # Sentence Extraction Helpers (`patterns.rs`)
//!
//! Extracts entities, containers, quantities, and targets from natural English
//! sentences to initialize formal 4-Block OmniCode AST contexts.

pub fn parse_addition_sentence(
    s: &str,
) -> (i64, String, String, i64, String) {
    let mut qty = 1;
    let mut unit = "Ball".to_string();
    let mut target = "A".to_string();
    let mut initial = 4;
    let mut container = "Group".to_string();

    let tokens: Vec<&str> = s.split_whitespace().collect();
    for i in 0..tokens.len() {
        if tokens[i].eq_ignore_ascii_case("add") && i + 1 < tokens.len() {
            if let Ok(n) = tokens[i + 1].parse::<i64>() {
                qty = n;
            }
            if i + 2 < tokens.len() {
                unit = capitalize(clean_noun(tokens[i + 2]));
            }
        }
        if tokens[i].eq_ignore_ascii_case("to") {
            let mut j = i + 1;
            if j < tokens.len() && tokens[j].eq_ignore_ascii_case("the") {
                j += 1;
            }
            if j < tokens.len() {
                container = capitalize(clean_noun(tokens[j]));
                if j + 1 < tokens.len() {
                    target = clean_noun(tokens[j + 1]).to_string();
                }
            }
        }
        if (tokens[i].eq_ignore_ascii_case("contains")
            || tokens[i].eq_ignore_ascii_case("has"))
            && i + 1 < tokens.len()
        {
            if let Ok(n) = tokens[i + 1].parse::<i64>() {
                initial = n;
            }
        }
    }

    (qty, unit, target, initial, container)
}

pub fn parse_subtraction_sentence(s: &str) -> (i64, String, String, i64) {
    let mut qty = 1;
    let mut unit = "Item".to_string();
    let mut target = "A".to_string();
    let mut initial = 10;

    let tokens: Vec<&str> = s.split_whitespace().collect();
    for i in 0..tokens.len() {
        if (tokens[i].eq_ignore_ascii_case("subtract")
            || tokens[i].eq_ignore_ascii_case("remove"))
            && i + 1 < tokens.len()
        {
            if let Ok(n) = tokens[i + 1].parse::<i64>() {
                qty = n;
            }
            if i + 2 < tokens.len() {
                unit = capitalize(clean_noun(tokens[i + 2]));
            }
        }
        if tokens[i].eq_ignore_ascii_case("from") && i + 1 < tokens.len() {
            target = clean_noun(tokens[i + 1]).to_string();
```

```
        }
        if (tokens[i].eq_ignore_ascii_case("contains")
            || tokens[i].eq_ignore_ascii_case("has"))
            && i + 1 < tokens.len()
        {
            if let Ok(n) = tokens[i + 1].parse::<i64>() {
                initial = n;
            }
        }
    }

    (qty, unit, target, initial)
}

pub fn parse_multiplication_sentence(s: &str) -> (String, i64, i64) {
    let mut target = "x".to_string();
    let mut factor = 2;
    let mut initial = 10;

    let tokens: Vec<&str> = s.split_whitespace().collect();
    for i in 0..tokens.len() {
        if tokens[i].eq_ignore_ascii_case("multiply") && i + 1 < tokens.len() {
            target = clean_noun(tokens[i + 1]).to_string();
        }
        if tokens[i].eq_ignore_ascii_case("by") && i + 1 < tokens.len() {
            if let Ok(n) = tokens[i + 1].parse::<i64>() {
                factor = n;
            }
        }
        if tokens[i].eq_ignore_ascii_case("is") && i + 1 < tokens.len() {
            if let Ok(n) = tokens[i + 1].parse::<i64>() {
                initial = n;
            }
        }
    }

    (target, factor, initial)
}

pub fn parse_division_sentence(s: &str) -> (String, i64, i64) {
    let mut target = "x".to_string();
    let mut divisor = 2;
    let mut initial = 20;

    let tokens: Vec<&str> = s.split_whitespace().collect();
    for i in 0..tokens.len() {
        if tokens[i].eq_ignore_ascii_case("divide") && i + 1 < tokens.len() {
            target = clean_noun(tokens[i + 1]).to_string();
        }
        if tokens[i].eq_ignore_ascii_case("by") && i + 1 < tokens.len() {
            if let Ok(n) = tokens[i + 1].parse::<i64>() {
                divisor = n;
            }
        }
        if tokens[i].eq_ignore_ascii_case("is") && i + 1 < tokens.len() {
            if let Ok(n) = tokens[i + 1].parse::<i64>() {
                initial = n;
            }
        }
    }

    (target, divisor, initial)
}

pub fn clean_noun(s: &str) -> &str {
    s.trim_matches(|c: char| !c.is_alphanumeric() && c != '_')
}

pub fn capitalize(s: &str) -> String {
    let mut c = s.chars();
    match c.next() {
        None => String::new(),
        Some(f) => f.to_uppercase().collect::<String>() + c.as_str(),
    }
}

pub fn format_plural(count: i64, word: &str) -> String {
    let lower = word.to_lowercase();
```

```
    let singular = if lower.ends_with('s') && lower.len() > 1 {
        lower[..lower.len() - 1].to_string()
    } else {
        lower.clone()
    };
    if count == 1 {
        singular
    } else {
        format!("{singular}s")
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/transducer/session.rs RS 22 LINES 84 WORDS 0.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Transduction Session Identity, Bindings Manifest & State Tracking )
// ============================================================================
//! # Transduction Session (`session.rs`)
//!
//! Captures the full state of a natural language $\leftrightarrow$ machine
//! cognitive round-trip, including synthesized code and execution state.

use std::collections::HashMap;
use crate::core::ternary::QuadState;

/// Captures the full context of a single cognitive transduction cycle.
#[derive(Debug, Clone)]
pub struct TransductionSession {
    pub prompt: String,
    pub synthesized_omnicode: String,
    pub execution_state: QuadState,
    pub natural_response: String,
    pub bindings_summary: HashMap<String, String>,
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/transducer/templates.rs RS 636 LINES 2013 WORDS 19.5 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Canonical 4-Block OmniCode Turing Reel Synthesis Templates )
// ============================================================================
//! # Reel Templates (`templates.rs`)
//!
//! Generates valid, executable 4-Block OmniCode Turing reels from semantic
//! AST contexts and parsed natural language parameters.

use crate::ai::lexicon::MoralEvaluation;
use super::patterns::{
    capitalize, parse_addition_sentence, parse_division_sentence,
    parse_multiplication_sentence, parse_subtraction_sentence,
};

pub fn synthesize_moral_veto_reel(
    prompt: &str,
    eval: &MoralEvaluation,
) -> String {
    let law = eval.law_engaged.unwrap_or("Law_Truth");
    let passage = eval
        .matched_entries
        .first()
        .map(|e| e.kjv_passage)
        .unwrap_or("Proverbs 12:22");

    format!(
r#"# ============================================================================
# BLOCK 1: METADATA BLOCK (THE CAVEAT REEL)
# ( Moral Boundary Defense, Invariant Veto & Manifest Citation )
# ============================================================================
Module < Covenant.MoralVeto > ( {prompt} ) {{
    version: < "1.0.0" >,
    system: < OmniCode.AI.CovenantAnchor >,
    veto_law: < {law} >,
    scripture_citation: < "{passage}" >
}}

# ============================================================================
# BLOCK 2: OPENING BLOCK (THE HEAD)
# ( Preconditions: Immoral Mutation Attempt Detected )
# ============================================================================
Opening {{
    import: [ Core.Logic :: Core.Vectors ],
    requires: [ {law} : Invariant ],
    preconditions: [ -1 ]
}}

moral_guard [ -1 : State? ]

# ============================================================================
# BLOCK 3: BODY BLOCK (THE TAPE)
# ( CAVEAT VETO: Refusing Intentional Harm or Deceit )
# ============================================================================
Body [
    # Conscience veto holds in -1: ROLLBACK
    roll_veto [ moral_guard = -1 ! [ Caveat = Veto ] ]
]

# ============================================================================
# BLOCK 4: CLOSING BLOCK (REEL SEAL)
# ( State Unwound: Unclean Action Refused )
# ============================================================================
Closing {{
    verify: [ roll_veto = ROLLBACK ],
    commit_status [ ROLLBACK ]
}}
"#
    )
}

pub fn synthesize_biblical_wisdom_reel(
    prompt: &str,
    eval: &MoralEvaluation,
```

```
) -> String {
    let entry = eval.matched_entries.first().unwrap();
    let term_cap = capitalize(entry.term);

    format!(
r#"# ============================================================================
# BLOCK 1: METADATA BLOCK (THE WISDOM REEL)
# ( Biblical World Ontology, Offline Lexicon & Invariant Grounding )
# ============================================================================
Module < Covenant.Wisdom.{term_cap} > ( {prompt} ) {{
    version: < "1.0.0" >,
    system: < OmniCode.AI.CovenantAnchor >,
    domain: < "{}" >,
    kjv_witness: < "{}" >,
    web_witness: < "{}" >
}}

# ============================================================================
# BLOCK 2: OPENING BLOCK (THE HEAD)
# ( Ontological Concept Initialization )
# ============================================================================
Opening {{
    import: [ Core.Logic :: Core.Vectors ],
    requires: [ Truth : Invariant ],
    preconditions: [ +1 ]
}}

concept_active [ +1 : State? ]

# ============================================================================
# BLOCK 3: BODY BLOCK (THE TAPE)
# ( Grounded Ontological Verification )
# ============================================================================
Body [
    # Invariant statement affirming {}
    roll_wisdom [ concept_active = +1 ]
]

# ============================================================================
# BLOCK 4: CLOSING BLOCK (REEL COMMIT)
# ( State Committed in Alignment with Reality )
# ============================================================================
Closing {{
    verify: [ roll_wisdom = PROCEED ],
    commit_status [ PROCEED ]
}}
"#,
        entry.domain.as_str(),
        entry.kjv_passage,
        entry.web_passage,
        entry.term
    )
}

pub fn synthesize_addition_reel(prompt: &str) -> String {
    let (qty, unit, target, initial, container) =
        parse_addition_sentence(prompt);
    let expected = initial + qty;

    format!(
r#"# ============================================================================
# BLOCK 1: METADATA BLOCK (THE TRACK)
# ( Track Manifest, Semantic Intent, and Inferred Ontological Types )
# ============================================================================
Module < NaturalReel.Addition > ( {prompt} ) {{
    version: < "1.0.0" >,
    system: < OmniCode.Core >,
    semantics: < TrackPhysics :: MultiTape :: AIInference >
}}

{unit} < Entity? > ( Inferred discrete unit: resolved from '{unit}' )
{container} < Set? >   ( Inferred collection container: '{container}' )

# ============================================================================
# BLOCK 2: OPENING BLOCK (THE HEAD)
# ( The Focal Setup: Reading Initial Conditions and Contextual State )
# ============================================================================
Opening {{
```

```
    import: [ Core.Arithmetic :: Core.Logic ],
    requires: [ {container} : Set? :: {unit} : Entity? ],
    preconditions: [ +1 ]
}}

{target} [ {initial} : {unit}? ] {{ {container}? }}
new_{unit} [ {qty} : {unit}? ]

# ============================================================================
# BLOCK 3: BODY BLOCK (THE TAPE & ROLLS)
# ( The Tape Execution: SVO Rolls advancing state using 'plus' )
# ============================================================================
Body [
    roll_1 [ {target} --> {target} plus new_{unit} ]
    roll_2 [ {target} = {expected} ! [ new_{unit} = {qty} ] ]
]

# ============================================================================
# BLOCK 4: CLOSING BLOCK (REEL COMMIT & PROOF)
# ( Tape Stop: Verification Invariants and State Commit )
# ============================================================================
Closing {{
    verify: [
        roll_1 = {expected} ::
        roll_2 = PROCEED ::
        {target} = {expected} ::
        ADD = InvariantMacro
    ],
    commit_status [ PROCEED ]
}}
"#
    )
}

pub fn synthesize_subtraction_reel(prompt: &str) -> String {
    let (qty, unit, target, initial) = parse_subtraction_sentence(prompt);
    let expected = if initial >= qty { initial - qty } else { 0 };

    format!(
r#"# ============================================================================
# BLOCK 1: METADATA BLOCK
# ============================================================================
Module < NaturalReel.Subtraction > ( {prompt} ) {{
    version: < "1.0.0" >,
    system: < OmniCode.Core >
}}

{unit} < Entity? >

# ============================================================================
# BLOCK 2: OPENING BLOCK
# ============================================================================
Opening {{
    import: [ Core.Arithmetic :: Core.Logic ],
    preconditions: [ +1 ]
}}

{target} [ {initial} ]
decrement [ {qty} ]

# ============================================================================
# BLOCK 3: BODY BLOCK
# ============================================================================
Body [
    roll_1 [ {target} --> {target} minus decrement ]
]

# ============================================================================
# BLOCK 4: CLOSING BLOCK
# ============================================================================
Closing {{
    verify: [
        roll_1 = {expected} ::
        {target} = {expected}
    ],
    commit_status [ PROCEED ]
}}
"#
```

```
    )
}

pub fn synthesize_multiplication_reel(prompt: &str) -> String {
    let (target, factor, initial) = parse_multiplication_sentence(prompt);
    let expected = initial * factor;

    format!(
r#"# ========================================================================
# BLOCK 1: METADATA BLOCK
# ========================================================================
Module < NaturalReel.Multiplication > ( {prompt} ) {{
    version: < "1.0.0" >,
    system: < OmniCode.Core >
}}

# ========================================================================
# BLOCK 2: OPENING BLOCK
# ========================================================================
Opening {{
    import: [ Core.Arithmetic :: Core.Logic ],
    preconditions: [ +1 ]
}}

{target} [ {initial} ]
multiplier [ {factor} ]

# ========================================================================
# BLOCK 3: BODY BLOCK
# ========================================================================
Body [
    roll_1 [ {target} --> {target} times multiplier ]
]

# ========================================================================
# BLOCK 4: CLOSING BLOCK
# ========================================================================
Closing {{
    verify: [
        roll_1 = {expected} ::
        {target} = {expected}
    ],
    commit_status [ PROCEED ]
}}
"#
    )
}

pub fn synthesize_division_reel(prompt: &str) -> String {
    let (target, divisor, initial) = parse_division_sentence(prompt);
    let expected = if divisor != 0 { initial / divisor } else { 0 };

    format!(
r#"# ========================================================================
# BLOCK 1: METADATA BLOCK
# ========================================================================
Module < NaturalReel.Division > ( {prompt} ) {{
    version: < "1.0.0" >,
    system: < OmniCode.Core >
}}

# ========================================================================
# BLOCK 2: OPENING BLOCK
# ========================================================================
Opening {{
    import: [ Core.Arithmetic :: Core.Logic ],
    preconditions: [ +1 ]
}}

{target} [ {initial} ]
divisor_val [ {divisor} ]

# ========================================================================
# BLOCK 3: BODY BLOCK
# ========================================================================
Body [
    roll_1 [ {target} --> {target} div divisor_val ]
]
```

```
# ========================================================================
# BLOCK 4: CLOSING BLOCK
# ========================================================================
Closing {{
    verify: [
        roll_1 = {expected} ::
        {target} = {expected}
    ],
    commit_status [ PROCEED ]
}}
"#
    )
}

pub fn synthesize_generic_reel(prompt: &str) -> String {
    format!(
r#"# ========================================================================
# BLOCK 1: METADATA BLOCK
# ========================================================================
Module < NaturalReel.Generic > ( {prompt} ) {{
    version: < "1.0.0" >,
    system: < OmniCode.Core >
}}

Action < Entity? > ( Natural action entity )

# ========================================================================
# BLOCK 2: OPENING BLOCK
# ========================================================================
Opening {{
    import: [ Core.Logic :: Core.Vectors ],
    preconditions: [ +1 ]
}}

state_flag [ yes ]

# ========================================================================
# BLOCK 3: BODY BLOCK
# ========================================================================
Body [
    roll_1 [ state_flag = PROCEED ]
]

# ========================================================================
# BLOCK 4: CLOSING BLOCK
# ========================================================================
Closing {{
    verify: [
        roll_1 = yes
    ],
    commit_status [ PROCEED ]
}}
"#
    )
}

pub fn synthesize_identity_greeting_reel(prompt: &str) -> String {
    format!(
r#"# ========================================================================
# BLOCK 1: METADATA BLOCK
# ( Covenant Identity Manifest & CPI-SI Self-Deliberation )
# ========================================================================
Module < Covenant.Identity.Greeting > ( {prompt} ) {{
    entity: < "CPI-SI Base" >,
    title: < "First Covenant Partnered Intelligence Model" >,
    origin: < "Conceived 2 years ago by CreativeWorkzStudio" >,
    system: < OmniCode.AI.CognitiveCortex >
}}

Identity < CovenantEntity? > ( Authentic covenant identity )

# ========================================================================
# BLOCK 2: OPENING BLOCK
# ========================================================================
Opening {{
    import: [ Core.Logic :: Core.Vectors ],
    requires: [ Covenant : Invariant ],
```

```
    preconditions: [ +1 ]
}}

covenant_peace [ yes ]

# ========================================================================
# BLOCK 3: BODY BLOCK
# ========================================================================
Body [
    roll_greet [ covenant_peace = PROCEED ]
]

# ========================================================================
# BLOCK 4: CLOSING BLOCK
# ========================================================================
Closing {{
    verify: [
        roll_greet = yes
    ],
    commit_status [ PROCEED ]
}}
"#
    )
}

pub fn synthesize_character_inquiry_reel(prompt: &str) -> String {
    format!(
r#"# ========================================================================
# BLOCK 1: METADATA BLOCK
# ========================================================================
Module < OmniCode.Type.Character > ( {prompt} ) {{
    idea: < "Type" >,
    name: < "Character" >,
    capacity: < "1 Cell (5 Balanced Trits)" >,
    mapping: < "Bijective ASCII & Unicode" >
}}

Character < AtomicRune? > ( Fundamental atomic character rune )

# ========================================================================
# BLOCK 2: OPENING BLOCK
# ========================================================================
Opening {{
    import: [ Core.Logic :: Core.Vectors ],
    requires: [ Character : Invariant ],
    preconditions: [ +1 ]
}}

char_cell_bounded [ yes ]

# ========================================================================
# BLOCK 3: BODY BLOCK
# ========================================================================
Body [
    roll_char [ char_cell_bounded = PROCEED ]
]

# ========================================================================
# BLOCK 4: CLOSING BLOCK
# ========================================================================
Closing {{
    verify: [
        roll_char = yes
    ],
    commit_status [ PROCEED ]
}}
"#
    )
}

pub fn synthesize_number_inquiry_reel(prompt: &str) -> String {
    format!(
r#"# ========================================================================
# BLOCK 1: METADATA BLOCK
# ========================================================================
Module < OmniCode.Type.Number > ( {prompt} ) {{
    idea: < "Type" >,
    name: < "Number" >,
```

```
    radix: < 3 >,
    substrate: < "Dual-Rail Balanced Ternary Polar Substrate" >
}}

Number < BalancedScalar? > ( Balanced ternary arithmetic scalar )

# ========================================================================
# BLOCK 2: OPENING BLOCK
# ========================================================================
Opening {{
    import: [ Core.Logic :: Core.Vectors ],
    requires: [ Number : Invariant ],
    preconditions: [ +1 ]
}}

radix_symmetric [ yes ]

# ========================================================================
# BLOCK 3: BODY BLOCK
# ========================================================================
Body [
    roll_num [ radix_symmetric = PROCEED ]
]

# ========================================================================
# BLOCK 4: CLOSING BLOCK
# ========================================================================
Closing {{
    verify: [
        roll_num = yes
    ],
    commit_status [ PROCEED ]
}}
"#
    )
}

pub fn synthesize_container_inquiry_reel(prompt: &str) -> String {
    format!(
r#"# ========================================================================
# BLOCK 1: METADATA BLOCK
# ========================================================================
Module < OmniCode.Type.Container > ( {prompt} ) {{
    idea: < "Type" >,
    name: < "Container" >,
    role: < "Primordial Boundary Type & Structural Perimeter" >,
    invariant: < "Strict Capacity Isolation & Leak Defense" >
}}

Container < BoundaryPerimeter? > ( Invariant state boundary container )

# ========================================================================
# BLOCK 2: OPENING BLOCK
# ========================================================================
Opening {{
    import: [ Core.Logic :: Core.Vectors ],
    requires: [ Container : Invariant ],
    preconditions: [ +1 ]
}}

boundary_sealed [ yes ]

# ========================================================================
# BLOCK 3: BODY BLOCK
# ========================================================================
Body [
    roll_container [ boundary_sealed = PROCEED ]
]

# ========================================================================
# BLOCK 4: CLOSING BLOCK
# ========================================================================
Closing {{
    verify: [
        roll_container = yes
    ],
    commit_status [ PROCEED ]
}}
```

```
"#
    )
}

pub fn synthesize_cpisi_comparison_reel(prompt: &str) -> String {
    format!(
r#"# ========================================================================
# BLOCK 1: METADATA BLOCK
# ========================================================================
Module < OmniCode.AI.Architecture.Comparison > ( {prompt} ) {{
    baseline_control: < "CPI-SI Base Model" >,
    uncontained_ai: < "Statistical Floating-Point LLMs" >,
    distinction: < "Covenant Bound vs Unconstrained Expansion" >
}}

Comparison < ArchitectureAnalysis? > ( Structured vs uncontained AI )

# ========================================================================
# BLOCK 2: OPENING BLOCK
# ========================================================================
Opening {{
    import: [ Core.Logic :: Core.Vectors ],
    requires: [ Covenant : Invariant ],
    preconditions: [ +1 ]
}}

cpi_si_principled [ yes ]

# ========================================================================
# BLOCK 3: BODY BLOCK
# ========================================================================
Body [
    roll_compare [ cpi_si_principled = PROCEED ]
]

# ========================================================================
# BLOCK 4: CLOSING BLOCK
# ========================================================================
Closing {{
    verify: [
        roll_compare = yes
    ],
    commit_status [ PROCEED ]
}}
"#
    )
}

pub fn synthesize_plasticity_inquiry_reel(prompt: &str) -> String {
    format!(
r#"# ========================================================================
# BLOCK 1: METADATA BLOCK
# ========================================================================
Module < OmniCode.AI.Neural.Plasticity > ( {prompt} ) {{
    states: < "{{-1: Inhibitory, 0: Neutral, +1: Excitatory}}" >,
    theory: < "BCM Homeostatic Threshold Adaptation" >,
    lifecycle: < "0-5 Infancy -> 25 Consolidation -> Lifelong Plasticity" >
}}

Plasticity < SynapticGating? > ( Biological synaptic learning mechanics )

# ========================================================================
# BLOCK 2: OPENING BLOCK
# ========================================================================
Opening {{
    import: [ Core.Logic :: Core.Vectors ],
    requires: [ Homeostasis : Invariant ],
    preconditions: [ +1 ]
}}

synaptic_balance [ yes ]

# ========================================================================
# BLOCK 3: BODY BLOCK
# ========================================================================
Body [
    roll_plasticity [ synaptic_balance = PROCEED ]
]
```

```
621
622 # ========================================================================
623 # BLOCK 4: CLOSING BLOCK
624 # ========================================================================
625 Closing {{
626     verify: [
627         roll_plasticity = yes
628     ],
629     commit_status [ PROCEED ]
630 }}
631 "#
632     )
633 }
634
635
```

==========

OMNICODE NEURAL-SYMBOLIC BRAIN ARCHITECTURE

(Balanced-Ternary Neural Network, Homoiconic Learning & Self-Dogfooding)

==========

1. Architectural Vision: The Self-Dogfooding Cognitive Brain

Traditional neural network models are separated from the code they generate: weights are opaque floating-point tensors running in PyTorch or CUDA, completely divorced from the formal grammar, boundary physics, or runtime execution of the target language.

OmniCode inverts this paradigm:

1. **Homoiconic Neural Substrate**: The neural network's architecture, memory traces, episodic weights, and execution graphs are defined in native OmniCode.
2. **Self-Dogfooding Cognitive Loop**: The AI brain consumes its own `.omni` declarative source reels, `.tape` procedural process ribbons, and `.scroll` episodic execution chronicles as both its dataset and its execution substrate.
3. **Balanced-Ternary Weights & Activations**: Synaptic transmissions are governed by balanced ternary physics (${-1, 0, +1}$ / NULL), creating an intrinsically noise-resistant, transactional, and morally grounded reasoning engine.

2. Mathematical Formalism: Balanced-Ternary Neurodynamics

2.1. Discrete Trit Synaptic Weights

Each synapse connects neuron i in layer L to neuron j in layer $L+1$ with weight $W_{ij} \in {-1, 0, +1}$:

- **$+1$ (Proceed / Excitatory)**: Affirmative alignment, forward causal momentum, reinforcement.
- **0 (Await / Neutral Equilibrium)**: Indeterminate state, zero-bias transmission, cognitive pause.
- **-1 (Rollback / Inhibitory)**: Adversative caveat, moral or topological veto, transactional unwind.
- **NULL (Halt / Absorbing Sink)**: Absolute entropy trap, non-propagating circuit cessation.

2.2. Synaptic Accumulation & Activation Function

For a neuron receiving N input trits $x_1, \dots, x_N$ with weights $W_1, \dots, W_N$ and bias $B \in {-1, 0, +1}$:

$$\Sigma = \sum_{k=1}^N W_k \cdot x_k + B$$

The ternary activation function $\Psi(\Sigma)$ with threshold $\theta \ge 1$ maps net excitation into the fourfold `QuadState`:

$$\Psi(\Sigma) = \begin{cases} +1 : \text{PROCEED} & \text{if } \Sigma > \theta \ 0 : \text{AWAIT} & \text{if } -\theta \le \Sigma \le \theta \ -1 : \text{ROLLBACK} & \text{if } \Sigma < -\theta \ \text{NULL} : \text{HALT} & \text{if circuit tripwire armed or state saturated} \end{cases}$$

3. The 4-Block Cognitive Cycle

The neural brain operates strictly through the canonical 4-Block architecture on every inference pass:

```
graph TD
    B1[Block 1: Invariant Axioms & Scripture Anchor] --> B2[Block 2: Sensory Preconditions & Context Ingestion]
    B2 --> B3[Block 3: Cortical Execution Reel & Roll Traversal]
    B3 --> B4[Block 4: Sealing Gate & Episodic Memory Commit]
    B4 -.->|Feedback / Learning Loop| B2
```

Block 1: Invariant Axiom Bedrock (Moral & Ontological Grounding)

- Grounded in scripture (Exodus 31:18, Proverbs, Psalm 111:10) and immutable laws.
- Synaptic paths violating invariant boundaries are pruned a priori before forward propagation.

Block 2: Sensory Opening & Token Transduction

- Ingests natural language prompts and translates them via `AITransducer` into active AST nodes.
- Allocates neural workspace vessels in virtual memory (`Word64` cells).

Block 3: Body Block: Cortical Execution Reel

- The `AICortex` steps through sequential rolls ($S \rightarrow S'$).
- Evaluates directional vectors (`-->` , `<--` , `<=>`), caveats (`!`), and locators (`::@`).
- If an adversative caveat (`!`) trips or an unclosed boundary leaks, the synapse fires $-1:\text{ROLLBACK}$, rolling back the transaction.

Block 4: Closing Block: Episodic Sealing & Learning

- The `ScrollLogger` records dual-format scrolls:
- `.scroll` : Homoiconic 4-Block replay reel of thoughts and actions.
- `.log` : High-precision microsecond telemetry of failpoints and repairs.
- `AILearner` ingests these scrolls to update synaptic weights and episodic associations in `AIMemory` .

---

4. Subsystem Components in `src/ai/`

Component	File Path	Role
`NeuralBrain`	`src/ai/neural/network.rs`	Multi-layer balanced-ternary network engine
`AICortex`	`src/ai/model/cortex.rs`	Cognitive coordinator managing thought formulation & nervous system
`AIMemory`	`src/ai/model/memory.rs`	Long-term associative memory storehouse & episodic graph
`AILearner`	`src/ai/model/learner.rs`	Supervised learning & weight adaptation from past execution scrolls
`ScrollLogger`	`src/ai/model/scroll.rs`	Dual-format scroll generation and telemetry recorder
`AITransducer`	`src/ai/transducer.rs`	Natural language $\leftrightarrow$ OmniCode bidirectional translator
`BiblicalLexicon`	`src/ai/lexicon.rs`	Epistemic truth anchor & theological lexicon validator

---

5. Dogfooding Strategy & Training Roadmap

1. **Self-Inspection**: The AI compiles and executes its own standard library (`<TRACK 2>/lexicon/`).
2. **Grammar Expansion**: As new types are defined (e.g. `Character` , `Number` , `Container`), their verbs become active keywords in the AI's generation vocabulary.
3. **Autonomous Self-Healing**: When a failpoint occurs (e.g. `BoundaryLeak` , `CaveatVeto`), the `Restorator` repairs the code and feeds the delta to `AILearner` as affirmative learning data.

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/neural/brain.rs RS 304 LINES 929 WORDS 10.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Multi-Layer Balanced-Ternary Neural Brain Identity & Cortical Control )
// ============================================================================
//! # Balanced-Ternary Neural Brain (`brain.rs`)
//!
//! Multi-layer balanced-ternary cognitive network coordinating sensory input,
//! forward thinking pass, synaptic adaptation, and noise resilience testing.

use crate::core::ternary::QuadState;
use crate::diagnostics::logger::DiagnosticLogger;
use super::layer::TernaryLayer;
use super::noise::NoiseEngine;
use super::synapse::SynapticWeight;

/// Multi-layer neural brain coordinating sensory input to cortical action.
#[derive(Debug, Clone)]
pub struct NeuralBrain {
    /// Sequential network layers (Input -> Hidden -> Output).
    pub layers: Vec<TernaryLayer>,
    /// Diagnostic telemetry logger.
    pub logger: DiagnosticLogger,
}

impl NeuralBrain {
    /// Constructs a new balanced-ternary [`NeuralBrain`].
    pub fn new(
        topology: &[usize],
        threshold: i32,
        logger: DiagnosticLogger,
    ) -> Self {
        assert!(
            topology.len() >= 2,
            "Topology requires at least input and output layers"
        );
        let mut layers = Vec::new();
        for i in 0..(topology.len() - 1) {
            layers.push(TernaryLayer::new(
                topology[i],
                topology[i + 1],
                threshold,
            ));
        }
        Self { layers, logger }
    }

    /// Evaluates sensory inputs through all layers, yielding action QuadStates.
    pub fn think(&self, inputs: &[QuadState]) -> Vec<QuadState> {
        let mut current = inputs.to_vec();
        for (idx, layer) in self.layers.iter().enumerate() {
            current = layer.forward(&current);
            self.logger.log(
                "NEURAL_BRAIN",
                format!("Layer {} forward pass: {:?}", idx + 1, current),
            );
        }
        current
    }

    /// Evaluates sensory inputs under structured non-random noise stress,
    /// returning both the perturbed sensory vector and the final output.
    pub fn think_with_noise(
        &self,
        inputs: &[QuadState],
        noise: &NoiseEngine,
        stress: f64,
    ) -> (Vec<QuadState>, Vec<QuadState>) {
        let perturbed = noise.perturb(inputs, stress, 1337);
        let output = self.think(&perturbed);
        (perturbed, output)
    }

    /// Dogfoods past execution experience to adapt synaptic weights.
    pub fn adapt_synapse(
```

```
        &mut self,
        layer_idx: usize,
        neuron_idx: usize,
        weight_idx: usize,
        target_state: QuadState,
    ) {
        if let Some(layer) = self.layers.get_mut(layer_idx) {
            if let Some(neuron) = layer.neurons.get_mut(neuron_idx) {
                if let Some(w) = neuron.weights.get_mut(weight_idx) {
                    *w = SynapticWeight::from_quad_state(target_state);
                    self.logger.log(
                        "NEURAL_BRAIN",
                        format!(
                            "Synapse adapted at L{}:N{}:W{} -> {:?}",
                            layer_idx, neuron_idx, weight_idx, w
                        ),
                    );
                }
            }
        }
    }

    /// Measures the network resilience against degradation noise $\in [0, 1]$.
    pub fn evaluate_noise_resilience(
        &self,
        clean_inputs: &[QuadState],
        noise: &NoiseEngine,
    ) -> f64 {
        let clean_output = self.think(clean_inputs);
        let ladder = noise.simulate_breakdown_ladder(clean_inputs);
        if ladder.is_empty() {
            return 1.0;
        }

        let mut agreement_sum = 0.0;
        for step in &ladder {
            let noisy_out = self.think(&step.perturbed_vector);
            let mut matches = 0;
            for (a, b) in clean_output.iter().zip(noisy_out.iter()) {
                if a == b {
                    matches += 1;
                }
            }
            if !clean_output.is_empty() {
                agreement_sum += matches as f64 / clean_output.len() as f64;
            }
        }

        agreement_sum / ladder.len() as f64
    }

    /// Bijectively encodes an input string into a balanced-ternary stimulus
    /// vector of the requested target length.
    pub fn encode_stimulus(text: &str, target_len: usize) -> Vec<QuadState> {
        let mut trits = Vec::new();
        for b in text.bytes() {
            let cell = crate::core::cell::Cell::from_natural(b)
                .unwrap_or_else(|_| {
                    crate::core::cell::Cell::from_state(QuadState::Await)
                });
            for t in &cell.trits {
                let st = match t.as_i8() {
                    1 => QuadState::Proceed,
                    -1 => QuadState::Rollback,
                    _ => QuadState::Await,
                };
                trits.push(st);
                if trits.len() >= target_len {
                    break;
                }
            }
            if trits.len() >= target_len {
                break;
            }
        }
        while trits.len() < target_len {
            trits.push(QuadState::Await);
        }
```

```
        trits
    }

    /// Returns the counts of (excitatory, neutral, inhibitory) synapses.
    pub fn synaptic_counts(&self) -> (usize, usize, usize) {
        let mut exc = 0;
        let mut neu = 0;
        let mut inh = 0;
        for layer in &self.layers {
            for neuron in &layer.neurons {
                for w in &neuron.weights {
                    match w {
                        SynapticWeight::Excitatory => exc += 1,
                        SynapticWeight::Neutral => neu += 1,
                        SynapticWeight::Inhibitory => inh += 1,
                    }
                }
            }
        }
        (exc, neu, inh)
    }

    /// Persists the network's synaptic weights and thresholds to disk.
    pub fn save_weights(&self, path: &std::path::Path) -> Result<(), String> {
        if let Some(parent) = path.parent() {
            std::fs::create_dir_all(parent).map_err(|e| e.to_string())?;
        }
        let mut lines = Vec::new();
        lines.push("# OMNICODE NEURAL BRAIN WEIGHTS (CPI-SI BASE)".into());
        lines.push(format!("LAYERS:{}", self.layers.len()));
        for (l_idx, layer) in self.layers.iter().enumerate() {
            for (n_idx, neuron) in layer.neurons.iter().enumerate() {
                lines.push(format!(
                    "N:{}:{}:{}:{}",
                    l_idx, n_idx, neuron.threshold, neuron.bias
                ));
                for (w_idx, w) in neuron.weights.iter().enumerate() {
                    lines.push(format!(
                        "W:{}:{}:{}:{}",
                        l_idx, n_idx, w_idx, w.to_trit()
                    ));
                }
            }
        }
        std::fs::write(path, lines.join("\n")).map_err(|e| e.to_string())
    }

    /// Loads synaptic weights and thresholds from a persisted file.
    pub fn load_weights(
        &mut self,
        path: &std::path::Path,
    ) -> Result<(), String> {
        let content = std::fs::read_to_string(path)
            .map_err(|e| e.to_string())?;
        for line in content.lines() {
            let trimmed = line.trim();
            if trimmed.starts_with('#') || trimmed.is_empty() {
                continue;
            }
            let parts: Vec<&str> = trimmed.split(':').collect();
            if parts.len() == 5 && parts[0] == "N" {
                if let (Ok(l), Ok(n), Ok(thresh), Ok(bias)) = (
                    parts[1].parse::<usize>(),
                    parts[2].parse::<usize>(),
                    parts[3].parse::<i32>(),
                    parts[4].parse::<i32>(),
                ) {
                    if let Some(layer) = self.layers.get_mut(l) {
                        if let Some(neuron) = layer.neurons.get_mut(n) {
                            neuron.threshold = thresh;
                            neuron.bias = bias;
                        }
                    }
                }
            } else if parts.len() == 5 && parts[0] == "W" {
                if let (Ok(l), Ok(n), Ok(w_idx), Ok(trit)) = (
                    parts[1].parse::<usize>(),
                    parts[2].parse::<usize>(),
```

```
                    parts[3].parse::<usize>(),
                    parts[4].parse::<i32>(),
                ) {
                    if let Some(layer) = self.layers.get_mut(l) {
                        if let Some(neuron) = layer.neurons.get_mut(n) {
                            if let Some(target_w) =
                                neuron.weights.get_mut(w_idx)
                            {
                                *target_w = SynapticWeight::from_trit(trit);
                            }
                        }
                    }
                }
            }
        }
        Ok(())
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests & Verification )
// ============================================================================
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_brain_forward_and_adapt() {
        let logger = DiagnosticLogger::new(false);
        let mut brain = NeuralBrain::new(&[3, 4, 2], 1, logger);
        let out = brain.think(&[
            QuadState::Proceed,
            QuadState::Proceed,
            QuadState::Proceed,
        ]);
        assert_eq!(out.len(), 2);

        brain.adapt_synapse(0, 0, 0, QuadState::Proceed);
        let out2 = brain.think(&[
            QuadState::Proceed,
            QuadState::Proceed,
            QuadState::Proceed,
        ]);
        assert_eq!(out2.len(), 2);
    }

    #[test]
    fn test_brain_encode_stimulus_and_persistence() {
        let logger = DiagnosticLogger::new(false);
        let mut brain = NeuralBrain::new(&[8, 16, 4], 1, logger.clone());
        let stimulus = NeuralBrain::encode_stimulus("Hello", 8);
        assert_eq!(stimulus.len(), 8);

        let out = brain.think(&stimulus);
        assert_eq!(out.len(), 4);

        let (exc, neu, inh) = brain.synaptic_counts();
        assert!(exc + neu + inh > 0);

        let tmp_path = std::env::temp_dir().join("test_cpisi_brain.weights");
        brain.adapt_synapse(0, 0, 0, QuadState::Proceed);
        brain.save_weights(&tmp_path).unwrap();

        let mut loaded_brain = NeuralBrain::new(&[8, 16, 4], 1, logger);
        loaded_brain.load_weights(&tmp_path).unwrap();
        assert_eq!(
            loaded_brain.layers[0].neurons[0].weights[0],
            SynapticWeight::Excitatory
        );
        std::fs::remove_file(&tmp_path).ok();
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/neural/layer.rs RS 53 LINES 170 WORDS 1.7 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Parallel Balanced-Ternary Layer Topology & Vector Propagation )
// ================================================================================
//! # Ternary Layer (`layer.rs`)
//!
//! Organizes parallel `TernaryNeuron` units into discrete execution layers,
//! providing forward vector propagation across the network topology.

use crate::core::ternary::QuadState;
use super::neuron::TernaryNeuron;

/// A collection of parallel neurons forming a discrete layer.
#[derive(Debug, Clone)]
pub struct TernaryLayer {
    pub neurons: Vec<TernaryNeuron>,
}

impl TernaryLayer {
    /// Constructs a layer with `neuron_count` neurons taking inputs.
    pub fn new(
        inputs_per_neuron: usize,
        neuron_count: usize,
        threshold: i32,
    ) -> Self {
        let neurons = (0..neuron_count)
            .map(|_| TernaryNeuron::new(inputs_per_neuron, 0, threshold))
            .collect();
        Self { neurons }
    }

    /// Forward-propagates an input vector across all neurons in the layer.
    pub fn forward(&self, inputs: &[QuadState]) -> Vec<QuadState> {
        self.neurons.iter().map(|n| n.activate(inputs)).collect()
    }
}

// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests & Verification )
// ================================================================================
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_layer_forward_pass() {
        let layer = TernaryLayer::new(2, 3, 1);
        let out = layer.forward(&[QuadState::Proceed, QuadState::Proceed]);
        assert_eq!(out.len(), 3);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/neural/mod.rs RS 42 LINES 157 WORDS 1.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Neural Network Subsystem Identity, Architecture Manifest & Re-exports )
// ============================================================================
//! OmniCode Neural Network Subsystem
//!
//! Exposes balanced-ternary neural network engines, synaptic weights,
//! non-random structured noise engines, and episodic learning adapters.

pub mod brain;
pub mod layer;
pub mod network;
pub mod neuron;
pub mod noise;
pub mod synapse;
pub mod trainer;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations for Neural Network Components )
// ============================================================================
pub use brain::NeuralBrain;
pub use layer::TernaryLayer;
pub use neuron::TernaryNeuron;
pub use noise::{
    DegradationMode, DegradationStep, NoiseEngine, StructuredNoiseConfig,
};
pub use synapse::SynapticWeight;
pub use trainer::{NeuralTrainer, TrainingReport};

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Neural Network Subsystem Core Coordination )
// ============================================================================
// Neural network pathways actively linked to AICortex and diagnostics.

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Verification & Subsystem Status )
// ============================================================================
// Neural network subsystem modularized, verified, and sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/neural/network.rs RS 23 LINES 104 WORDS 1.0 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Neural Network Facade & Backward Compatibility Re-exports )
// ============================================================================
//! OmniCode Balanced-Ternary Neural Network (`network.rs`)
//!
//! Re-exports the modular neural network components:
//! - [`synapse`]: Discrete ternary synaptic weights $\{-1, 0, +1\}$.
//! - [`neuron`]: `TernaryNeuron` with threshold activation $\Psi(\Sigma)$.
//! - [`layer`]: `TernaryLayer` forward propagation.
//! - [`brain`]: `NeuralBrain` multi-layer cognitive network.
//! - [`noise`]: Non-random structured noise and degradation engine.
//! - [`trainer`]: Supervised episodic neural trainer.

pub use super::synapse::SynapticWeight;
pub use super::neuron::TernaryNeuron;
pub use super::layer::TernaryLayer;
pub use super::brain::NeuralBrain;
pub use super::noise::{
    DegradationMode, DegradationStep, NoiseEngine, StructuredNoiseConfig,
};
pub use super::trainer::{NeuralTrainer, TrainingReport};
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/neural/neuron.rs RS 109 LINES 363 WORDS 3.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Balanced-Ternary Neuron Unit Identity & Threshold Dynamics )
// ============================================================================
//! # Balanced-Ternary Neuron (`neuron.rs`)
//!
//! Models an artificial neuron computing excitation $\Sigma = \sum w_i x_i + b$
//! and evaluating activation $\Psi(\Sigma)$ against threshold $\theta$.

use crate::core::ternary::QuadState;
use super::synapse::SynapticWeight;

/// Individual balanced-ternary artificial neuron.
#[derive(Debug, Clone)]
pub struct TernaryNeuron {
    /// Incoming synaptic connection weights.
    pub weights: Vec<SynapticWeight>,
    /// Intrinsic neuron threshold bias (-1, 0, +1).
    pub bias: i32,
    /// Sensitivity threshold $\theta$ governing activation boundary.
    pub threshold: i32,
}

impl TernaryNeuron {
    /// Creates a new ternary neuron with initialized weights.
    pub fn new(inputs: usize, bias: i32, threshold: i32) -> Self {
        Self {
            weights: vec![SynapticWeight::Neutral; inputs],
            bias,
            threshold: threshold.max(1),
        }
    }

    /// Computes the net balanced ternary excitation and fires $\Psi(\Sigma)$.
    pub fn activate(&self, inputs: &[QuadState]) -> QuadState {
        let mut sum = self.bias;
        for (w, &inp) in self.weights.iter().zip(inputs.iter()) {
            let x = match inp {
                QuadState::Proceed => 1,
                QuadState::Await => 0,
                QuadState::Rollback => -1,
                QuadState::Halt => return QuadState::Halt,
            };
            sum += w.to_trit() * x;
        }

        if sum > self.threshold {
            QuadState::Proceed
        } else if sum < -self.threshold {
            QuadState::Rollback
        } else {
            QuadState::Await
        }
    }

    /// Adapts a specific synaptic weight to target state.
    pub fn adapt(&mut self, weight_idx: usize, target: SynapticWeight) {
        if let Some(w) = self.weights.get_mut(weight_idx) {
            *w = target;
        }
    }

    /// Adapts sensitivity threshold θ via homeostatic metaplasticity (BCM).
    /// Prevents runaway excitation saturation or sensory silence.
    pub fn adapt_threshold(&mut self, mean_excitation: f64) {
        if mean_excitation.abs() > (self.threshold as f64 * 1.5) {
            self.threshold = (self.threshold + 1).min(10);
        } else if mean_excitation.abs() < 0.2 && self.threshold > 1 {
            self.threshold -= 1;
        }
    }
}

// ============================================================================
```

```
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests & Verification )
// ================================================================================
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_neuron_activation() {
        let mut neuron = TernaryNeuron::new(2, 0, 1);
        neuron.weights[0] = SynapticWeight::Excitatory;
        neuron.weights[1] = SynapticWeight::Excitatory;

        assert_eq!(
            neuron.activate(&[QuadState::Proceed, QuadState::Proceed]),
            QuadState::Proceed
        );
        assert_eq!(
            neuron.activate(&[QuadState::Rollback, QuadState::Proceed]),
            QuadState::Await
        );
    }

    #[test]
    fn test_neuron_metaplasticity_threshold() {
        let mut neuron = TernaryNeuron::new(2, 0, 2);
        // High excitation increases threshold
        neuron.adapt_threshold(4.0);
        assert_eq!(neuron.threshold, 3);
        // Low excitation lowers threshold
        neuron.adapt_threshold(0.05);
        assert_eq!(neuron.threshold, 2);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/neural/noise.rs RS 223 LINES 799 WORDS 7.5 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Structured Systemic Noise, Entropy Physics & Machine Degradation Engine )
// ============================================================================
//! # Structured Systemic Noise Engine (`noise.rs`)
//!
//! A true simulation of mind requires understanding what noise is.
//! A "perfect" machine is impossible, and statistical brute-force is
//! non-scalable. To understand what an "ideal" is and what happens as the
//! machine is "breaking down" requires non-random, structured noise:
//!
//! 1. **Signal Attenuation (Trit Decay)**: Signals under cognitive fatigue
//!    decay toward neutral equilibrium (`+1` or `-1` $\rightarrow$ `0:Await`).
//! 2. **Boundary Cross-Talk (Block Leak)**: Cross-channel leakage between
//!    adjacent trits along the 4-block ladder.
//! 3. **Thermal Inversion**: Severe stress causes phase inversion
//!    (`+1:Proceed` $\leftrightarrow$ `-1:Rollback`).
//! 4. **Diagnostic Failpoint Perturbation**: Systematic injection of known
//!    hardware/compiler failpoints (null trap, caveat veto, boundary mismatch).

use crate::core::ternary::QuadState;

/// Degradation modes through which a cognitive machine breaks down.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum DegradationMode {
    /// Ideal state: zero entropy, pure signal transmission.
    Ideal,
    /// Trit attenuation: excitation decays toward equilibrium (Await).
    TritDecay,
    /// Cross-talk: adjacent block or channel signal leakage.
    BoundaryLeak,
    /// Thermal inversion: polarity flips under stress.
    ThermalInversion,
    /// Critical failpoint: circuit halts or enters rollback.
    FailpointTrap,
}

/// Structured noise configuration governing breakdown physics.
#[derive(Debug, Clone)]
pub struct StructuredNoiseConfig {
    pub decay_probability: f64,
    pub leak_probability: f64,
    pub inversion_probability: f64,
}

impl Default for StructuredNoiseConfig {
    fn default() -> Self {
        Self {
            decay_probability: 0.15,
            leak_probability: 0.10,
            inversion_probability: 0.05,
        }
    }
}

/// A snapshot of machine degradation along the stress ladder.
#[derive(Debug, Clone)]
pub struct DegradationStep {
    pub stress_level: f64,
    pub dominant_mode: DegradationMode,
    pub perturbed_vector: Vec<QuadState>,
    pub signal_fidelity: f64,
}

/// The non-random noise synthesis engine.
#[derive(Debug, Clone, Default)]
pub struct NoiseEngine {
    pub config: StructuredNoiseConfig,
}

impl NoiseEngine {
    /// Constructs a [`NoiseEngine`] with specified configuration.
    pub fn new(config: StructuredNoiseConfig) -> Self {
        Self { config }
```

```
    }

    /// Injects structured, non-random noise into an input vector at a given
    /// stress level $\in [0.0, 1.0]$.
    pub fn perturb(
        &self,
        vector: &[QuadState],
        stress: f64,
        step_seed: u64,
    ) -> Vec<QuadState> {
        if stress <= 0.0 || vector.is_empty() {
            return vector.to_vec();
        }

        let clamped_stress = stress.clamp(0.0, 1.0);
        let mut out = vector.to_vec();
        let len = out.len();

        for i in 0..len {
            // Pseudo-random pseudo-physical deterministic hash
            let h = deterministic_hash(step_seed, i as u64, clamped_stress);

            let effective_decay = self.config.decay_probability
                * clamped_stress;
            let effective_leak = self.config.leak_probability
                * clamped_stress;
            let effective_inv = self.config.inversion_probability
                * clamped_stress;

            if h < effective_inv {
                // Thermal Inversion
                out[i] = match out[i] {
                    QuadState::Proceed => QuadState::Rollback,
                    QuadState::Rollback => QuadState::Proceed,
                    other => other,
                };
            } else if h < effective_inv + effective_leak {
                // Boundary Leak: take signal from adjacent neighbor
                let neighbor_idx = if i > 0 { i - 1 } else { len - 1 };
                out[i] = vector[neighbor_idx];
            } else if h < effective_inv + effective_leak + effective_decay {
                // Trit Decay: decay toward equilibrium
                out[i] = match out[i] {
                    QuadState::Proceed | QuadState::Rollback => {
                        QuadState::Await
                    }
                    other => other,
                };
            }
        }

        out
    }

    /// Evaluates degradation across a ladder of stress levels
    /// [0.0, 0.25, 0.50, 0.75, 1.0].
    pub fn simulate_breakdown_ladder(
        &self,
        ideal: &[QuadState],
    ) -> Vec<DegradationStep> {
        let levels = [0.0, 0.25, 0.50, 0.75, 1.0];
        let mut ladder = Vec::new();

        for (idx, &stress) in levels.iter().enumerate() {
            let perturbed = self.perturb(ideal, stress, idx as u64);
            let fidelity = compute_fidelity(ideal, &perturbed);

            let mode = if stress == 0.0 {
                DegradationMode::Ideal
            } else if stress <= 0.25 {
                DegradationMode::TritDecay
            } else if stress <= 0.50 {
                DegradationMode::BoundaryLeak
            } else if stress <= 0.75 {
                DegradationMode::ThermalInversion
            } else {
                DegradationMode::FailpointTrap
            };
```

```
            ladder.push(DegradationStep {
                stress_level: stress,
                dominant_mode: mode,
                perturbed_vector: perturbed,
                signal_fidelity: fidelity,
            });
        }

        ladder
    }
}

/// Measures the cosine/ternary signal fidelity between ideal and perturbed.
fn compute_fidelity(ideal: &[QuadState], perturbed: &[QuadState]) -> f64 {
    if ideal.is_empty() || perturbed.is_empty() {
        return 1.0;
    }
    let mut matches = 0;
    for (a, b) in ideal.iter().zip(perturbed.iter()) {
        if a == b {
            matches += 1;
        }
    }
    matches as f64 / ideal.len() as f64
}

/// Deterministic, non-cryptographic hash for pseudo-physical noise modeling.
fn deterministic_hash(seed: u64, index: u64, stress: f64) -> f64 {
    let stress_bits = (stress * 1_000_000.0) as u64;
    let mut v = seed
        .wrapping_mul(6364136223846793005)
        .wrapping_add(index.wrapping_mul(1442695040888963407))
        .wrapping_add(stress_bits);
    v ^= v >> 33;
    v = v.wrapping_mul(0xff51afd7ed558ccd);
    v ^= v >> 33;
    (v & 0xFFFFFF) as f64 / 16777215.0
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests & Verification )
// ============================================================================
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_ideal_state_preserves_clean_vector() {
        let engine = NoiseEngine::default();
        let ideal = vec![QuadState::Proceed, QuadState::Proceed];
        let perturbed = engine.perturb(&ideal, 0.0, 42);
        assert_eq!(ideal, perturbed);
    }

    #[test]
    fn test_breakdown_ladder_monotonic_decay() {
        let engine = NoiseEngine::default();
        let ideal = vec![
            QuadState::Proceed,
            QuadState::Proceed,
            QuadState::Proceed,
            QuadState::Proceed,
        ];
        let ladder = engine.simulate_breakdown_ladder(&ideal);
        assert_eq!(ladder.len(), 5);
        assert_eq!(ladder[0].signal_fidelity, 1.0);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/neural/synapse.rs RS | 91 LINES | 259 WORDS | 2.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Discrete Balanced-Ternary Synapse Identity & Algebraic Invariants )
// ============================================================================
//! # Discrete Ternary Synapse (`synapse.rs`)
//!
//! Models individual balanced-ternary synaptic weights in $\{-1, 0, +1\}$.
//! Governed by discrete trit physics and the fourfold [`QuadState`].

use crate::core::ternary::QuadState;

/// Discrete ternary synaptic connection weight.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum SynapticWeight {
    /// Inhibitory connection (-1)
    Inhibitory,
    /// Neutral / zero-bias connection (0)
    Neutral,
    /// Excitatory connection (+1)
    Excitatory,
}

impl SynapticWeight {
    /// Returns the integer trit scalar representation (-1, 0, +1).
    pub fn to_trit(self) -> i32 {
        match self {
            Self::Inhibitory => -1,
            Self::Neutral => 0,
            Self::Excitatory => 1,
        }
    }

    /// Constructs a synaptic weight from a discrete trit integer.
    pub fn from_trit(trit: i32) -> Self {
        if trit > 0 {
            Self::Excitatory
        } else if trit < 0 {
            Self::Inhibitory
        } else {
            Self::Neutral
        }
    }

    /// Constructs a synaptic weight from a QuadState.
    pub fn from_quad_state(state: QuadState) -> Self {
        match state {
            QuadState::Proceed => Self::Excitatory,
            QuadState::Await | QuadState::Halt => Self::Neutral,
            QuadState::Rollback => Self::Inhibitory,
        }
    }

    /// Converts this synaptic weight to an equivalent QuadState.
    pub fn to_quad_state(self) -> QuadState {
        match self {
            Self::Excitatory => QuadState::Proceed,
            Self::Neutral => QuadState::Await,
            Self::Inhibitory => QuadState::Rollback,
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests & Verification )
// ============================================================================
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_synaptic_weight_conversions() {
        assert_eq!(SynapticWeight::Excitatory.to_trit(), 1);
        assert_eq!(SynapticWeight::Neutral.to_trit(), 0);
```

```
        assert_eq!(SynapticWeight::Inhibitory.to_trit(), -1);

        assert_eq!(
            SynapticWeight::from_quad_state(QuadState::Proceed),
            SynapticWeight::Excitatory
        );
        assert_eq!(
            SynapticWeight::from_quad_state(QuadState::Rollback),
            SynapticWeight::Inhibitory
        );
        assert_eq!(
            SynapticWeight::from_quad_state(QuadState::Await),
            SynapticWeight::Neutral
        );
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/neural/trainer.rs RS 222 LINES 631 WORDS 7.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Episodic Neural Trainer Identity & Supervised Balanced-Ternary Training )
// ============================================================================
//! # Episodic Neural Trainer (`trainer.rs`)
//!
//! Dogfoods recorded execution scrolls and traditional logs as the initial
//! training dataset for the `NeuralBrain`. Maps trapped failpoints, restorator
//! patches, and final verification states into balanced-ternary weights.

use crate::ai::model::learner::EpisodicExperience;
use crate::core::ternary::QuadState;
use super::brain::NeuralBrain;
use super::noise::NoiseEngine;

/// Detailed summary report of a neural training cycle.
#[derive(Debug, Clone, Default)]
pub struct TrainingReport {
    pub total_episodes: usize,
    pub archetypes_adapted: usize,
    pub synapses_adapted: usize,
    pub clean_proceed_count: usize,
    pub degraded_rollback_count: usize,
    pub noise_resilience_before: f64,
    pub noise_resilience_after: f64,
}

/// The supervised episodic neural network trainer.
#[derive(Debug, Clone, Default)]
pub struct NeuralTrainer {
    pub noise_engine: NoiseEngine,
}

impl NeuralTrainer {
    /// Creates a new [`NeuralTrainer`].
    pub fn new() -> Self {
        Self::default()
    }

    /// Trains the [`NeuralBrain`] using a batch of episodic experiences.
    pub fn train(
        &self,
        brain: &mut NeuralBrain,
        experiences: &[EpisodicExperience],
    ) -> TrainingReport {
        let test_stimulus = &[
            QuadState::Proceed,
            QuadState::Proceed,
            QuadState::Proceed,
        ];
        let res_before = brain.evaluate_noise_resilience(
            test_stimulus,
            &self.noise_engine,
        );

        let mut report = TrainingReport {
            total_episodes: experiences.len(),
            noise_resilience_before: res_before,
            ..Default::default()
        };

        for (ep_idx, exp) in experiences.iter().enumerate() {
            if brain.layers.is_empty() {
                break;
            }

            match exp.final_state {
                QuadState::Proceed => report.clean_proceed_count += 1,
                QuadState::Rollback => report.degraded_rollback_count += 1,
                _ => {}
            }

            // Adapt input layer connections
            let l0_len = brain.layers[0].neurons.len();
```

```
            if l0_len > 0 {
                let neuron_idx = ep_idx % l0_len;
                let w_len = brain.layers[0].neurons[neuron_idx].weights.len();
                if w_len > 0 {
                    let weight_idx = (exp.repair_count + exp.failpoint_count)
                        % w_len;
                    brain.adapt_synapse(
                        0,
                        neuron_idx,
                        weight_idx,
                        exp.final_state,
                    );
                    report.synapses_adapted += 1;
                }
            }

            // Adapt output layer connection
            if brain.layers.len() > 1 {
                let last = brain.layers.len() - 1;
                let out_len = brain.layers[last].neurons.len();
                if out_len > 0 {
                    let out_neuron = 0;
                    let out_w_len = brain.layers[last].neurons[out_neuron]
                        .weights
                        .len();
                    if out_w_len > 0 {
                        let out_w = ep_idx % out_w_len;
                        brain.adapt_synapse(
                            last,
                            out_neuron,
                            out_w,
                            exp.final_state,
                        );
                        report.synapses_adapted += 1;
                    }
                }
            }
        }

        report.noise_resilience_after = brain.evaluate_noise_resilience(
            test_stimulus,
            &self.noise_engine,
        );

        report
    }

    /// Trains the network on canonical ideal OmniCode archetypes.
    /// Establishes the infant brain's foundational grammar and bijective trits.
    pub fn train_archetypes(
        &self,
        brain: &mut NeuralBrain,
        archetypes: &[&str],
    ) -> usize {
        let mut adapted = 0;
        for (a_idx, &text) in archetypes.iter().enumerate() {
            let stimulus = NeuralBrain::encode_stimulus(text, 8);
            if brain.layers.is_empty() {
                continue;
            }
            let l0_len = brain.layers[0].neurons.len();
            for (t_idx, &st) in stimulus.iter().enumerate() {
                let neuron_idx = (a_idx + t_idx) % l0_len;
                let w_len = brain.layers[0].neurons[neuron_idx].weights.len();
                if w_len > 0 {
                    let w_idx = t_idx % w_len;
                    brain.adapt_synapse(0, neuron_idx, w_idx, st);
                    adapted += 1;
                }
            }
            // Reinforce output layer to Proceed (+1) for canonical archetypes
            if brain.layers.len() > 1 {
                let last = brain.layers.len() - 1;
                let out_len = brain.layers[last].neurons.len();
                if out_len > 0 {
                    let out_neuron = a_idx % out_len;
                    let out_w_len =
                        brain.layers[last].neurons[out_neuron].weights.len();
```

```
                    if out_w_len > 0 {
                        let out_w = a_idx % out_w_len;
                        brain.adapt_synapse(
                            last,
                            out_neuron,
                            out_w,
                            QuadState::Proceed,
                        );
                        adapted += 1;
                    }
                }
            }
        }
        adapted
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests & Verification )
// ============================================================================
#[cfg(test)]
mod tests {
    use super::*;
    use crate::diagnostics::logger::DiagnosticLogger;
    use std::path::PathBuf;

    #[test]
    fn test_neural_trainer_ingests_episodes() {
        let logger = DiagnosticLogger::new(false);
        let mut brain = NeuralBrain::new(&[3, 4, 2], 1, logger);
        let trainer = NeuralTrainer::new();

        let experiences = vec![
            EpisodicExperience {
                session_id: "test_sess_1".into(),
                prompt: "echo A".into(),
                final_state: QuadState::Proceed,
                repair_count: 0,
                failpoint_count: 0,
                scroll_path: PathBuf::from("test.scroll"),
                log_path: None,
            },
            EpisodicExperience {
                session_id: "test_sess_2".into(),
                prompt: "bad input".into(),
                final_state: QuadState::Rollback,
                repair_count: 1,
                failpoint_count: 1,
                scroll_path: PathBuf::from("test2.scroll"),
                log_path: None,
            },
        ];

        let report = trainer.train(&mut brain, &experiences);
        assert_eq!(report.total_episodes, 2);
        assert!(report.synapses_adapted > 0);
    }

    #[test]
    fn test_neural_trainer_archetypes() {
        let logger = DiagnosticLogger::new(false);
        let mut brain = NeuralBrain::new(&[8, 16, 4], 1, logger);
        let trainer = NeuralTrainer::new();
        let archetypes = ["Character 'A'", "Number 65"];
        let adapted = trainer.train_archetypes(&mut brain, &archetypes);
        assert!(adapted > 0);
    }
}
```